

Trabalho Parcial

Serviço de Estoque e Serviço de Pedidos: duas aplicações integradas, com testes unitários e testes de integração

256

testes unitários

100%

de cobertura

21

testes de integração

PROFESSOR

Carlos Alves da Silva

INTEGRANTES

[Nome do(a) integrante 1]

[Nome do(a) integrante 2]

Sumário

1 Introdução	2
1.1 Atendimento ao enunciado da atividade	2
2 Documentação das aplicações	3
2.1 Visão geral	3
2.2 Aplicação 1: Serviço de Estoque	4
2.3 Aplicação 2: Serviço de Pedidos	5
2.4 Como executar	6
2.5 Tecnologias	7
2.6 Limitações conhecidas	7
3 Especificação de requisitos	8
3.1 Aplicação 1: Serviço de Estoque	8
3.2 Aplicação 2: Serviço de Pedidos	9
3.3 Requisitos não funcionais (ambas as aplicações)	9
4 Plano e estratégia de testes	10
4.1 Objetivo	10
4.2 Níveis de teste	10
4.3 Técnicas de projeto de casos de teste aplicadas	10
4.4 Documentação dos testes	11
4.5 Critérios de aceitação	11
4.6 Ambiente de teste	11
4.7 Como executar os testes	12
5 Casos de teste: objetivo de cada teste	13
5.1 Resumo	13
5.2 Testes unitários: Aplicação 1: Estoque	13
5.3 Testes unitários: Aplicação 2: Pedidos	19
5.4 Testes de integração: Estoque + Pedidos	26
6 Evidências de execução	28
6.1 Resumo	28
6.2 Testes unitários e cobertura	28
6.3 Testes de integração	31
6.4 Execução funcional pelo terminal (CLIs)	34
6.5 Execução funcional pela API (requisições e respostas)	37
7 Matriz de rastreabilidade	40
8 Conclusão	43
Apêndice A: Saída completa dos testes unitários	44
Apêndice B: Arquivos da entrega e execução	49

1 Introdução

Este relatório documenta o **Trabalho Parcial** da disciplina de Teste e Requisitos de Software. Foram construídas **duas aplicações** em Python que simulam o back-office de uma loja de informática: a **Aplicação 1, Serviço de Estoque**, que controla produtos e saldos, e a **Aplicação 2, Serviço de Pedidos**, que registra vendas e depende do estoque para reservar mercadorias. As aplicações rodam em processos separados, se comunicam por uma API REST (HTTP/JSON) e são operadas pelo terminal, por meio de uma interface de linha de comando (CLI) em cada aplicação.

O trabalho foi guiado por requisitos: cada requisito funcional, regra de negócio e requisito não funcional recebeu um identificador (Capítulo 3). Os testes foram projetados com técnicas de caixa-preta e caixa-branca (partição de equivalência, análise de valor limite, tabela de decisão, transição de estados, dublês de teste e injeção de falhas). Cada teste documenta seu objetivo e os requisitos que verifica, o que permite construir a matriz de rastreabilidade do Capítulo 7.

256/256

testes unitários aprovados

100%

cobertura dos testes
unitários (linhas + ramos;
meta $\geq 90\%$)

21/21

testes de integração
aprovados

30/30

requisitos com teste
rastreado

1.1 Atendimento ao enunciado da atividade

Item solicitado	Como foi atendido	Onde
Duas aplicações em qualquer linguagem	Serviço de Estoque e Serviço de Pedidos, em Python 3 + Flask (somente back-end), executados como processos independentes que se comunicam por HTTP e operados pelo terminal por meio de interfaces de linha de comando (CLI).	Cap. 2
Documentação sobre o que a aplicação faz	Arquitetura, funcionalidades, endpoints, regras de negócio, configuração e forma de execução de cada aplicação. Especificação de requisitos com 30 itens identificados.	Cap. 2 e 3
Documentação sobre o objetivo de cada teste	188 funções de teste (277 casos executados), cada uma com <i>objetivo</i> , <i>técnica</i> , <i>requisitos verificados</i> e resultado.	Cap. 4 e 5
Testes unitários com cobertura de pelo menos 90%	256 testes unitários, com cobertura de 100% das linhas e 100% dos ramos . O comando usa <code>--cov-fail-under=90</code> e falha se a meta não for atingida.	Cap. 6.2
Testes de integração entre as aplicações	21 testes que sobem as duas aplicações em processos separados e verificam fluxos completos, atômidade, concorrência e falhas de rede (queda, timeout, porta fechada).	Cap. 6.3
PDF com evidências	Este documento: saídas reais do pytest, relatórios de cobertura, capturas de tela e registro das chamadas HTTP.	Cap. 6
Código-fonte em RAR/ZIP	<code>codigo_fonte.zip</code> (aplicações, testes, documentação e scripts).	Apêndice B
Vídeo da execução funcional	<code>video_execucao.mp4</code> : quatro terminais reais (dois servidores e duas CLIs) operando as aplicações, seguidos da execução dos testes.	Apêndice B

2 Documentação das aplicações

2.1 Visão geral

O projeto simula o back-office de uma pequena loja de informática e é composto por **duas aplicações independentes**, escritas em Python 3 com o micro-framework Flask, que se comunicam por **HTTP/JSON**:

	Aplicação 1: Serviço de Estoque	Aplicação 2: Serviço de Pedidos
Pacote	<code>estoque_service/</code>	<code>pedidos_service/</code>
Porta padrão	5001	5002
Responsabilidade	Cadastro de produtos e controle de saldo	Registro de vendas (pedidos) e cancelamentos
Dados	Produtos (SKU, nome, preço, saldo, estoque mínimo)	Pedidos (cliente, itens, totais, status)
Depende de	nada	Serviço de Estoque (via <code>ESTOQUE_URL</code>)
Interface do usuário	CLI no terminal: <code>python -m estoque_service.cli</code>	CLI no terminal: <code>python -m pedidos_service.cli</code>

O Serviço de Pedidos **não tem acesso direto aos dados do estoque**: toda consulta de catálogo, reserva (baixa) e devolução de produtos é feita chamando a API REST do Serviço de Estoque. É essa comunicação que os testes de integração verificam.

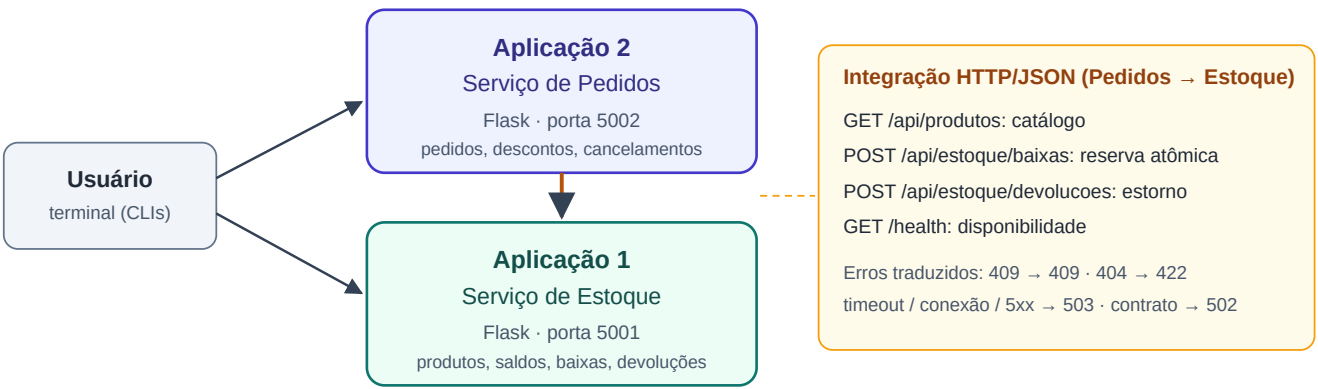


Figura 1: Visão geral da arquitetura. A Aplicação 2 consome a API REST da Aplicação 1.

Arquitetura interna (igual nas duas aplicações)

Cada aplicação segue uma arquitetura em camadas, o que permite testar cada parte isoladamente:

Camada	Estoque	Pedidos	Função
HTTP (Flask)	<code>app.py</code>	<code>app.py</code>	Rotas, conversão JSON e mapeamento de erros para códigos HTTP
Serviço (casos de uso)	<code>servico.py</code>	<code>servico.py</code>	Orquestra as regras e garante atomicidade e exclusão mútua
Domínio	<code>dominio.py</code>	<code>dominio.py</code>	Entidades, validações e regras de negócio puras

Camada	Estoque	Pedidos	Função
Repositório	repositorio.py	repositorio.py	Armazenamento em memória (substituível por banco de dados)
Integração	n/a	cliente_estoque.py	Cliente HTTP do estoque: traduz respostas e falhas de rede em exceções
Interface (CLI)	cli.py	cli.py	Menu interativo no terminal que consome a API REST da própria aplicação

As dependências são **injetadas pelo construtor** (`criar_app(servico)`, `ServicoPedidos(cliente_estoque, repositorio, relógio)`). Nos testes unitários elas são trocadas por duplões de teste. Nos testes de integração são usadas as implementações reais.

2.2 Aplicação 1: Serviço de Estoque

O que faz

Mantém o catálogo de produtos da loja e o saldo de cada um. Permite:

1. **Cadastrar produtos** com SKU (código único), nome, preço, quantidade inicial e estoque mínimo.
2. **Consultar** a lista de produtos ou um produto pelo SKU.
3. **Registrar entradas** (reposição de mercadoria).
4. **Baixar itens em lote** (reservar mercadoria para um pedido). A operação é **atômica**: se qualquer item não tiver saldo, nenhum item é baixado. A resposta informa o nome e o **preço vigente** de cada item.
5. **Devolver itens** (estorno de um pedido cancelado).
6. **Listar produtos que precisam de reposição** (saldo $\leq$ estoque mínimo).

Endpoints

Método	Rota	Descrição	Respostas
GET	/health	Verificação de saúde	200
GET	/api/produtos	Lista produtos (ordenados por SKU)	200
POST	/api/produtos	Cadastra produto	201, 400, 409
GET	/api/produtos/{sku}	Consulta produto	200, 400, 404
POST	/api/produtos/{sku}/entradas	Registra entrada {"quantidade": n}	200, 400, 404
GET	/api/produtos/reposicao	Produtos com saldo $\leq$ mínimo	200
POST	/api/estoque/baixas	Baixa em lote {"itens": [{"sku", "quantidade"}]}	200, 400, 404, 409
POST	/api/estoque/devolucoes	Devolução em lote (mesmo formato)	200, 400, 404

Exemplo de baixa:

```
POST /api/estoque/baixas
{"itens": [{"sku": "TEC-001", "quantidade": 2}]}

200 OK
{"itens": [{"sku": "TEC-001", "nome": "Teclado Mecânico", "preco": "250.00", "quantidade": 2, "saldo": 8}]}
```

Exemplo de erro padronizado (saldo insuficiente):

```
409 CONFLICT
{"erro": "Estoque insuficiente para 'TEC-001': disponível 8, solicitado 50.",
"codigo": "ESTOQUE_INSUFICIENTE", "sku": "TEC-001", "disponivel": 8, "solicitado": 50}
```

Regras de negócio principais

- SKU único, com 3 a 20 caracteres (letras, números e hífen), normalizado para maiúsculas.
- Nome obrigatório (até 100 caracteres). Preço maior que zero e até R\$ 1.000.000,00, arredondado em 2 casas (arredondamento comercial).
- Quantidades por operação são inteiras, de 1 a 10.000. O saldo nunca fica negativo e a capacidade máxima por produto é 100.000.
- Baixas e devoluções em lote são **tudo ou nada**. SKUs repetidos no lote são somados.
- Todas as operações de escrita são protegidas por *lock*, de modo que requisições simultâneas não vendem além do saldo.

2.3 Aplicação 2: Serviço de Pedidos

O que faz

Registra as vendas da loja. Para cada pedido:

1. **Valida** o cliente e os itens (quantidade, SKU, limites).
2. **Chama o Serviço de Estoque** para reservar (baixar) todos os itens de uma vez. O estoque devolve o **preço oficial** de cada produto, e o preço que o cliente eventualmente enviar é ignorado.
3. **Calcula os totais** com **desconto progressivo**:

Subtotal	Desconto
abaixo de R\$ 500,00	0%
de R\$ 500,00 a R\$ 999,99	5%
a partir de R\$ 1.000,00	10%

4. **Registra o pedido** com status `CONFIRMADO`.

Também permite **cancelar** um pedido confirmado, o que devolve os itens ao estoque e muda o status para `CANCELADO`, além de **listar/consultar pedidos**, **exibir o catálogo** (obtido do estoque) e **informar a saúde** da dependência.

Endpoints

Método	Rota	Descrição	Respostas
GET	<code>/health</code>	Saúde do serviço e da dependência (estoque)	200
GET	<code>/api/catalogo</code>	Catálogo vindo do Serviço de Estoque	200, 502, 503
GET	<code>/api/pedidos</code>	Lista pedidos (mais recente primeiro)	200
POST	<code>/api/pedidos</code>	Cria pedido {"cliente", "itens": [{"sku", "quantidade"}]}	201, 400, 409, 422, 502, 503
GET	<code>/api/pedidos/{id}</code>	Consulta pedido	200, 404

Método	Rota	Descrição	Respostas
POST	/api/pedidos/{id}/cancelamento	Cancela pedido e devolve itens ao estoque	200, 404, 409, 503

Tratamento das respostas do estoque

Situação no Serviço de Estoque	Resposta do Serviço de Pedidos
Baixa realizada	201 Created (pedido confirmado)
Saldo insuficiente (409 ESTOQUE_INSUFICIENTE)	409 Conflict, sem pedido registrado
Produto inexistente (404 PRODUTO_NAO_ENCONTRADO)	422 Unprocessable Entity
Estoque fora do ar, recusa de conexão, timeout ou 5xx	503 Service Unavailable
Resposta fora do contrato (JSON inválido, campos ausentes)	502 Bad Gateway

Se o estoque estiver indisponível durante um **cancelamento**, o pedido **permanece CONFIRMADO**. Assim os dois serviços nunca ficam inconsistentes (o pedido não é cancelado sem que a mercadoria volte ao estoque).

Configuração (variáveis de ambiente)

Variável	Padrão	Uso
ESTOQUE_HOST / ESTOQUE_PORTA	127.0.0.1 / 5001	Endereço do Serviço de Estoque
PEDIDOS_HOST / PEDIDOS_PORTA	127.0.0.1 / 5002	Endereço do Serviço de Pedidos
ESTOQUE_URL	http://127.0.0.1:5001	URL que o Pedidos usa para chamar o Estoque
ESTOQUE_TIMEOUT	3	Tempo limite (segundos) das chamadas ao Estoque

2.4 Como executar

Pré-requisito: Python 3.10 ou superior.

```
python -m venv .venv
# Linux/macOS: source .venv/bin/activate | Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt

# Terminal 1: Aplicação 1
python -m estoque_service # http://127.0.0.1:5001

# Terminal 2: Aplicação 2
python -m pedidos_service # http://127.0.0.1:5002
```

Com os dois servidores no ar, abra mais dois terminais e use as interfaces de linha de comando:

```
# Terminal 3: CLI do Estoque (cadastrar, listar, registrar entrada, reposição)
python -m estoque_service.cli

# Terminal 4: CLI de Pedidos (catálogo, criar, listar, cancelar, situação dos serviços)
python -m pedidos_service.cli
```

Exemplo de uso da CLI de Pedidos:

```
=====
SERVIÇO DE PEDIDOS  (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 2

Cliente: Maria Souza
Informe os itens (SKU em branco para finalizar).
SKU: TEC-001
Quantidade: 2
SKU: MOU-001
Quantidade: 1
SKU:
OK: pedido confirmado!
Pedido #1 - Maria Souza - CONFIRMADO
TEC-001   Teclado mecânico      2 x   R$ 250,00 =   R$ 500,00
MOU-001   Mouse sem fio        1 x   R$ 120,00 =   R$ 120,00
Subtotal R$ 620,00 | Desconto 5% (-R$ 31,00) | TOTAL R$ 589,00
```

As CLIs também aceitam as variáveis `ESTOQUE_URL` e `PEDIDOS_URL` para apontar para outro endereço. Erros da API aparecem com o código HTTP e a mensagem, por exemplo `ERRO (HTTP 409): Estoque insuficiente para 'MON-001': disponível 2, solicitado 5`.

2.5 Tecnologias

- **Python 3.11** e **Flask 3** (APIs REST); interface de linha de comando com a biblioteca padrão (`input/print`)
- **requests** (cliente HTTP entre as aplicações)
- **pytest**, **pytest-cov** (coverage.py com cobertura de ramos) e **pytest-html** (testes e relatórios)
- **Playwright** e **xterm.js** (somente para gravar o vídeo dos terminais e gerar o PDF de evidências)

2.6 Limitações conhecidas

- Os dados ficam em memória: reiniciar um serviço apaga seus dados. O repositório foi isolado em uma classe própria justamente para permitir a troca por um banco de dados.
- Se a conexão cair **depois** de o estoque efetivar uma baixa mas **antes** de o Pedidos receber a resposta, a mercadoria fica reservada sem pedido. Em produção isso seria resolvido com chaves de idempotência ou com o padrão *Saga*. O cenário está fora do escopo do trabalho.

3 Especificação de requisitos

Os requisitos foram levantados antes da implementação e cada um recebeu um identificador único. Esses identificadores aparecem no campo **Requisitos** de cada caso de teste e na **matriz de rastreabilidade**, o que permite verificar que todo requisito é coberto por pelo menos um teste.

Convenção: **RF** = requisito funcional, **RN** = regra de negócio, **RNF** = requisito não funcional. O sufixo **E** indica o Serviço de Estoque e **P** o Serviço de Pedidos.

3.1 Aplicação 1: Serviço de Estoque

Requisitos funcionais

ID	Descrição
RF-E01	Cadastrar produto com SKU, nome, preço, quantidade inicial e estoque mínimo.
RF-E02	Listar os produtos e consultar um produto pelo SKU.
RF-E03	Registrar entrada (reposição) de mercadoria em um produto.
RF-E04	Baixar (reservar) itens em lote, informando nome e preço vigente de cada item.
RF-E05	Devolver itens ao estoque (estorno de pedido cancelado).
RF-E06	Listar os produtos que precisam de reposição.
RF-E07	Oferecer uma interface de linha de comando (menu no terminal) para cadastrar, listar, registrar entradas e consultar a reposição, exibindo os erros da API ao usuário.

Regras de negócio

ID	Descrição
RN-E01	O SKU é obrigatório, único, tem de 3 a 20 caracteres (A–Z, 0–9 e hífen, sem começar com hífen) e é normalizado para maiúsculas.
RN-E02	O nome é obrigatório e tem no máximo 100 caracteres.
RN-E03	O preço deve ser numérico, maior que zero e no máximo R\$ 1.000.000,00, arredondado para 2 casas (half-up).
RN-E04	Quantidades por operação são inteiras entre 1 e 10.000. O saldo nunca fica negativo e a capacidade máxima por produto é 100.000 unidades.
RN-E05	Um produto precisa de reposição quando o saldo é menor ou igual ao estoque mínimo.
RN-E06	Baixas e devoluções em lote são atômicas (tudo ou nada), aceitam até 50 itens e somam SKUs repetidos.

3.2 Aplicação 2: Serviço de Pedidos

Requisitos funcionais

ID	Descrição
RF-P01	Exibir o catálogo de produtos, obtido do Serviço de Estoque.
RF-P02	Criar pedido informando cliente e itens (SKU e quantidade).
RF-P03	Listar os pedidos e consultar um pedido pelo número.
RF-P04	Cancelar pedido, devolvendo os itens ao Serviço de Estoque.
RF-P05	Informar a saúde do serviço e a disponibilidade do Serviço de Estoque.
RF-P06	Oferecer uma interface de linha de comando (menu no terminal) para ver o catálogo, criar, listar e cancelar pedidos e consultar a situação dos serviços.

Regras de negócio

ID	Descrição
RN-P01	O cliente é obrigatório (3 a 80 caracteres). O pedido tem de 1 a 20 produtos distintos, com quantidade inteira de 1 a 100 por produto. Itens com o mesmo SKU são agrupados.
RN-P02	O preço unitário é sempre o informado pelo Serviço de Estoque (fonte da verdade), nunca o enviado pelo cliente.
RN-P03	Desconto progressivo sobre o subtotal: 0% abaixo de R\$ 500,00, 5% de R\$ 500,00 a R\$ 999,99 e 10% a partir de R\$ 1.000,00, arredondado para 2 casas.
RN-P04	O pedido só é confirmado se o estoque reservar todos os itens. Se houver falta de saldo ou produto inexistente, nenhum pedido é registrado.
RN-P05	Somente pedidos CONFIRMADOS podem ser cancelados. Os itens são devolvidos ao estoque antes da mudança de status, e se o estoque estiver indisponível o pedido permanece CONFIRMADO.
RN-P06	Falhas de comunicação com o estoque (conexão recusada, timeout, erro 5xx) resultam em HTTP 503, e respostas fora do contrato resultam em HTTP 502.

3.3 Requisitos não funcionais (ambas as aplicações)

ID	Descrição
RNF01	As APIs seguem o estilo REST com JSON. Erros usam o formato padronizado {"erro", "codigo"} e códigos HTTP adequados.
RNF02	Os testes unitários devem cobrir pelo menos 90% do código (linhas e ramos), verificado automaticamente.
RNF03	Chamadas do Pedidos ao Estoque têm tempo limite configurável (padrão de 3 segundos).
RNF04	Operações simultâneas não podem deixar o estoque negativo nem devolver itens em duplicidade (thread-safety).
RNF05	Portas, endereços e a URL do estoque são configuráveis por variáveis de ambiente.

4 Plano e estratégia de testes

4.1 Objetivo

Demonstrar, com evidências automatizadas, que as duas aplicações atendem aos requisitos especificados:

- individualmente, com **testes unitários** que cobrem pelo menos 90% do código (meta atingida: 100% de linhas e de ramos);
- em conjunto, com **testes de integração** que exercitam a comunicação HTTP real entre o Serviço de Pedidos e o Serviço de Estoque.

4.2 Níveis de teste

Nível	Pasta	O que é testado	Isolamento	Ferramentas
Unitário	tests/unit/estoque/	Domínio, repositório, serviço, rotas HTTP e CLI do Estoque	Sem rede: Flask <i>test client</i> e objetos em memória	pytest, pytest-cov
Unitário	tests/unit/pedidos/	Domínio, repositório, serviço, cliente HTTP, rotas e CLI do Pedidos	O Serviço de Estoque é substituído por dublês de teste	pytest, unittest.mock, pytest-cov
Integração	tests/integration/	As duas aplicações rodando juntas (inclusive usadas pelas CLIs, como um usuário no terminal)	Nenhum dublê: cada aplicação roda em seu próprio processo , em portas TCP reais	pytest, subprocess, requests

Por que os testes de integração usam processos separados

A fixture `ProcessoServico` (`tests/integration/conftest.py`) executa `python -m estoque_service` e `python -m pedidos_service` como processos do sistema operacional, cada um em uma porta livre. O Pedidos recebe a URL do Estoque pela variável `ESTOQUE_URL`, como aconteceria em produção. Com isso os testes validam, além da lógica, a **configuração**, a **serialização JSON**, os **códigos HTTP** e o comportamento diante de **falhas reais de rede**: processo derrubado, porta fechada e servidor que não responde.

4.3 Técnicas de projeto de casos de teste aplicadas

Técnica	Onde foi aplicada (exemplos)
Partição de equivalência	Classes válidas/inválidas de SKU, nome, preço, quantidade, cliente e corpo JSON.
Análise de valor limite	SKU com 2/3/20/21 caracteres. Preço 0,00/0,01/1.000.000,00/1.000.000,01. Quantidade 0/1/100/101 e 0/1/10.000/10.001. Desconto em 499,99/500,00/999,99/1.000,00. Estoque mínimo (saldo = mínimo ± 1). Capacidade máxima + 1. Lote com 50/51 itens e pedido com 20/21 produtos.
Tabela de decisão	Tradução de cada código de erro do estoque (409, 404, 400, 4xx desconhecido, 5xx) na exceção e no status HTTP do Pedidos.
Transição de estados	Pedido CONFIRMADO → CANCELADO (válida) e CANCELADO → CANCELADO (inválida, 409).

Técnica	Onde foi aplicada (exemplos)
Dublês de teste	<i>Stub</i> (RespostaFalsa), <i>Mock</i> (unittest.mock.Mock com verificação de chamadas), <i>Fake</i> e <i>Spy</i> (EstoqueFalso, estoque em memória que registra as chamadas). Nas CLIs, <i>SessaoFlask</i> (fake de requests.Session que encaminha as chamadas ao app em memória) e <i>roteiro</i> (simula o usuário digitando).
Injeção de falhas	Timeout, conexão recusada, erro 5xx, JSON inválido e contrato violado (unitário). Porta fechada, servidor mudo e queda do processo do Estoque (integração).
Teste de concorrência	30 baixas simultâneas (unitário) e 25 pedidos e 10 cancelamentos simultâneos via HTTP (integração), com threading.Barrier.
Injeção de dependência	Relógio, repositório e cliente de estoque injetados no ServicoPedidos. Serviço injetado no criar_app().

Todos os testes seguem o padrão **AAA (Arrange, Act, Assert)** e são **independentes entre si**: cada teste cria seus próprios dados, e nos testes de integração cada teste usa SKUs únicos gerados com `uuid`.

4.4 Documentação dos testes

Cada função de teste possui uma *docstring* padronizada com três campos:

```
def test_baixa_e_atomica_quando_um_item_nao_tem_saldo(self, servico):
    """Objetivo: Garantir que, se um item do lote não tem saldo, nenhum item é baixado (tudo ou nada).
    Técnica: Teste de transação/atomicidade com valor limite (saldo+1)
    Requisitos: RN-E06, RN-P04
    """
```

O script `scripts/gerar_documentacao_testes.py` lê essas *docstrings* e gera automaticamente o catálogo de casos de teste (`docs/04-casos-de-teste.md`) e a matriz de rastreabilidade (`docs/05-matriz-de-rastreabilidade.md`), com o resultado da última execução. O script **falha** se algum teste não tiver objetivo documentado ou citar um requisito inexistente, o que mantém documentação e código sincronizados.

4.5 Critérios de aceitação

Critério	Como é verificado
100% dos testes unitários e de integração passando	Saída do pytest e relatórios HTML/JUnit
Cobertura dos testes unitários $\geq 90\%$ (linhas + ramos)	<code>pytest --cov --cov-fail-under=90</code> (o comando falha se a meta não for atingida)
Todo requisito coberto por pelo menos um teste	Matriz de rastreabilidade

4.6 Ambiente de teste

- Python 3.11 (compatível com 3.10+), Linux ou Windows
- pytest 8.3, pytest-cov 5.0 (coverage.py 7), pytest-html 4.1
- Serviços executados localmente em `127.0.0.1` com portas escolhidas dinamicamente

4.7 Como executar os testes

```
pip install -r requirements-dev.txt

# Testes unitários + relatório de cobertura (falha se < 90%)
pytest tests/unit --cov --cov-report=term-missing --cov-report=html --cov-fail-under=90

# Testes de integração (sobem as duas aplicações automaticamente)
pytest tests/integration -v

# Tudo
pytest
```

O relatório HTML de cobertura é gerado em `htmlcov/index.html`.

5 Casos de teste: objetivo de cada teste

Documento gerado automaticamente por `scripts/gerar_documentacao_testes.py` a partir das *docstrings* dos testes. O campo **Casos** indica quantas execuções o teste gera (testes parametrizados executam uma vez por conjunto de dados).

5.1 Resumo

Nível	Aplicação	Funções de teste	Casos executados	Resultado
Unitário	Aplicação 1: Estoque	85	135	135/135 aprovados
Unitário	Aplicação 2: Pedidos	84	121	121/121 aprovados
Integração	Estoque + Pedidos	19	21	21/21 aprovados
Total		188	277	

5.2 Testes unitários: Aplicação 1: Estoque

`tests/unit/estoque/test_dominio.py`

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E01	<code>test_normaliza_para_maiusculas_e_remove_espacos</code> <code>TestValidarSku</code>	Verificar que o SKU informado com espaços e letras minúsculas é normalizado.	Partição de equivalência (classe válida)	RN-E01	1	✓ Passou (1/1)
UT-E02	<code>test_aceita_skus_nos_limites_validos</code> <code>TestValidarSku</code>	Garantir que SKUs com 3 e 20 caracteres (limites) e formatos válidos são aceitos.	Análise de valor limite (mínimo e máximo válidos)	RN-E01	4	✓ Passou (4/4)
UT-E03	<code>test_rejeita_skus_fora_do_padrao</code> <code>TestValidarSku</code>	Garantir que SKUs curtos (2), longos (21), vazios ou com caracteres inválidos são rejeitados.	Valor limite (mínimo-1, máximo+1) e partição de equivalência (classes inválidas)	RN-E01	7	✓ Passou (7/7)
UT-E04	<code>test_rejeita_sku_que_nao_e_texto</code> <code>TestValidarSku</code>	Verificar que um SKU ausente ou de tipo diferente de texto é rejeitado.	Partição de equivalência (tipo inválido)	RN-E01	3	✓ Passou (3/3)
UT-E05	<code>test_remove_espacos_das_pontas</code> <code>TestValidarNome</code>	Verificar que o nome válido é aceito sem os espaços das extremidades.	Partição de equivalência (classe válida)	RN-E02	1	✓ Passou (1/1)
UT-E06	<code>test_aceita_nome_com_tamanho_maximo</code> <code>TestValidarNome</code>	Garantir que um nome com exatamente 100 caracteres é aceito.	Análise de valor limite (máximo)	RN-E02	1	✓ Passou (1/1)
UT-E07	<code>test_rejeita_nome_acima_do_tamanho_maximo</code> <code>TestValidarNome</code>	Garantir que um nome com 101 caracteres é rejeitado.	Análise de valor limite (máximo+1)	RN-E02	1	✓ Passou (1/1)
UT-E08	<code>test_rejeita_nome_vazio_ou_invalido</code> <code>TestValidarNome</code>	Verificar que nome ausente, vazio, só com espaços ou não textual é rejeitado.	Partição de equivalência (classes inválidas)	RN-E02	4	✓ Passou (4/4)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E09	test_aceita_precos_validos_e_normaliza_duas_casas TestValidarPreco	Verificar que preços em texto, inteiro, float e Decimal são aceitos e normalizados para 2 casas, incluindo os limites 0,01 e 1.000.000,00.	Partição de equivalência e análise de valor limite	RN-E03	7	✓ Passou (7/7)
UT-E10	test_arredonda_meio_centavo_para_cima TestValidarPreco	Garantir o arredondamento comercial (half-up) na terceira casa decimal.	Análise de valor limite (ponto de arredondamento)	RN-E03	1	✓ Passou (1/1)
UT-E11	test_rejeita_preco_zero_ou_negativo TestValidarPreco	Garantir que preços iguais a zero, negativos ou que arredondam para zero são rejeitados.	Análise de valor limite (mínimo-1 centavo)	RN-E03	4	✓ Passou (4/4)
UT-E12	test_rejeita_preco_acima_do_maximo TestValidarPreco	Garantir que um preço de 1.000.000,01 (máximo + 1 centavo) é rejeitado.	Análise de valor limite (máximo+1)	RN-E03	1	✓ Passou (1/1)
UT-E13	test_rejeita_preco_nao_numerico TestValidarPreco	Verificar que valores ausentes, booleanos, listas, textos não numéricos, NaN e infinito são rejeitados.	Partição de equivalência (classes inválidas)	RN-E03	7	✓ Passou (7/7)
UT-E14	test_aceita_quantidades_dentro_do_intervalo TestValidarQuantidade	Verificar que quantidades de 1 a 10.000 (inclusive os limites) são aceitas.	Análise de valor limite (mínimo, nominal, máximo)	RN-E04	3	✓ Passou (3/3)
UT-E15	test_rejeita_quantidades_fora_do_intervalo TestValidarQuantidade	Garantir que as quantidades 0, negativas e 10.001 são rejeitadas com mensagem adequada.	Análise de valor limite (mínimo-1, máximo+1)	RN-E04	3	✓ Passou (3/3)
UT-E16	test_rejeita_quantidade_nao_inteira TestValidarQuantidade	Verificar que booleanos, floats, textos e valor ausente não são aceitos como quantidade.	Partição de equivalência (tipos inválidos)	RN-E04	4	✓ Passou (4/4)
UT-E17	test_minimo_configuravel_aceita_zero_e_informa_o_campo TestValidarQuantidade	Verificar que o mínimo pode ser 0 (saldo inicial) e que a mensagem cita o campo validado.	Análise de valor limite (mínimo configurável)	RN-E04	1	✓ Passou (1/1)
UT-E18	test_criar_com_dados_validos_normaliza_campos TestProduto	Verificar que um produto é criado a partir de um dicionário válido, com SKU, nome e preço normalizados.	Partição de equivalência (classe válida)	RF-E01, RN-E01, RN-E02, RN-E03	1	✓ Passou (1/1)
UT-E19	test_criar_usa_zero_como_padrao_para_saldo_e_minimo TestProduto	Garantir que quantidade e estoque mínimo são opcionais e assumem 0.	Partição de equivalência (campos opcionais omitidos)	RF-E01	1	✓ Passou (1/1)
UT-E20	test_criar_rejeita_dados_que_nao_sao_objeto TestProduto	Verificar que dados que não são um objeto JSON (dicionário) são rejeitados.	Partição de equivalência (tipo inválido)	RF-E01, RNF01	3	✓ Passou (3/3)
UT-E21	test_precisa_reposicao_quando_saldo_menor_ou_igual_ao_minimo TestProduto	Verificar o alerta de reposição com saldo abaixo (2), igual (3) e acima (4) do mínimo 3.	Análise de valor limite em torno do estoque mínimo	RN-E05	3	✓ Passou (3/3)
UT-E22	test_adicionar_incrementa_saldo TestProduto	Verificar que uma entrada soma a quantidade ao saldo atual.	Partição de equivalência (classe válida)	RF-E03	1	✓ Passou (1/1)
UT-E23	test_adicionar_ate_a_capacidade_maxima_e_permitido TestProduto	Garantir que é possível atingir exatamente a capacidade máxima de armazenamento.	Análise de valor limite (capacidade máxima)	RN-E04	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E24	test_adicionar_acima_da_capacidade_maxima_e_rejeitado_sem_alterar_saldo TestProduto	Garantir que uma entrada que ultrapassa a capacidade em 1 unidade é rejeitada e o saldo não muda.	Análise de valor limite (capacidade+1)	RN-E04	1	✓ Passou (1/1)
UT-E25	test_remover_decrescenta_saldo TestProduto	Verificar que uma baixa subtrai a quantidade do saldo.	Partição de equivalência (classe válida)	RF-E04	1	✓ Passou (1/1)
UT-E26	test_remover_exatamente_o_saldo_zera_o_estoque TestProduto	Garantir que é possível baixar exatamente todo o saldo (saldo final 0).	Análise de valor limite (quantidade = saldo)	RF-E04, RN-E04	1	✓ Passou (1/1)
UT-E27	test_remover_mais_que_o_saldo_lanca_estoque_insuficiente TestProduto	Garantir que baixar saldo+1 lança EstoqueInsuficiente com os dados do erro e não altera o saldo.	Análise de valor limite (saldo+1)	RN-E04	1	✓ Passou (1/1)
UT-E28	test_para_dict_serializa_preco_como_texto_com_duas_casas TestProduto	Verificar o formato JSON do produto (preço em texto com 2 casas e indicador de reposição).	Verificação de contrato de saída	RF-E02, RNF01	1	✓ Passou (1/1)
UT-E29	test_execoes_informam_codigo_e_mensagem TestExcecoes	Verificar que as exceções de negócio expõem código padronizado e mensagem legível.	Verificação de contrato de erro	RNF01	1	✓ Passou (1/1)

tests/unit/estoque/test_repositorio_e_servico.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E30	test_adicionar_e_obter_produto TestRepositorio	Verificar que um produto adicionado ao repositório pode ser recuperado pelo SKU.	Partição de equivalência (classe válida)	RF-E01, RF-E02	1	✓ Passou (1/1)
UT-E31	test_adicionar_sku_duplicado_lanca_erro TestRepositorio	Garantir que o repositório não aceita dois produtos com o mesmo SKU.	Partição de equivalência (classe inválida: duplicidade)	RN-E01	1	✓ Passou (1/1)
UT-E32	test_obter_sku_inexistente_lanca_erro TestRepositorio	Verificar que consultar um SKU inexistente lança ProdutoNaoEncontrado.	Partição de equivalência (classe inválida)	RF-E02	1	✓ Passou (1/1)
UT-E33	test_listar_retorna_produtos_ordenados_por_sku TestRepositorio	Verificar que a listagem é ordenada alfabeticamente pelo SKU.	Verificação de ordenação	RF-E02	1	✓ Passou (1/1)
UT-E34	test_agrupa_skus_repetidos_somando_quantidades TestNormalizarItens	Verificar que itens com o mesmo SKU (inclusive com grafias diferentes) são agrupados e somados.	Partição de equivalência (classe válida com repetição)	RN-E06	1	✓ Passou (1/1)
UT-E35	test_rejeita_lista_vazia_ou_ausente TestNormalizarItens	Garantir que a operação exige uma lista com pelo menos um item.	Análise de valor limite (0 itens) e partição de equivalência (tipo inválido)	RN-E06	4	✓ Passou (4/4)
UT-E36	test_aceita_lote_com_numero_maximo_de_itens TestNormalizarItens	Garantir que um lote com exatamente 50 itens é aceito.	Análise de valor limite (máximo)	RN-E06	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E37	test_rejeita_lote_acima_do_numero_maximo_de_itens TestNormalizarItens	Garantir que um lote com 51 itens é rejeitado.	Análise de valor limite (máximo+1)	RN-E06	1	✓ Passou (1/1)
UT-E38	test_rejeita_item_que_nao_e_objeto TestNormalizarItens	Verificar que cada item precisa ser um objeto com sku e quantidade.	Partição de equivalência (tipo inválido)	RN-E06, RNF01	1	✓ Passou (1/1)
UT-E39	test_rejeita_soma_agrupada_acima_do_limite_por_operacao TestNormalizarItens	Garantir que o limite de 10.000 unidades vale para a soma dos itens agrupados (5.000 + 5.001).	Análise de valor limite após agrupamento	RN-E04, RN-E06	1	✓ Passou (1/1)
UT-E40	test_cadastrar_produto_o_torna_disponivel_na_listagem TestCadastroEConsulta	Verificar que produtos cadastrados aparecem na listagem e na consulta por SKU.	Partição de equivalência (classe válida)	RF-E01, RF-E02	1	✓ Passou (1/1)
UT-E41	test_cadastrar_sku_duplicado_com_outra_grafia_e_rejeitado TestCadastroEConsulta	Garantir a unicidade do SKU independentemente de maiúsculas/minúsculas.	Partição de equivalência (classe inválida: duplicidade)	RN-E01	1	✓ Passou (1/1)
UT-E42	test_obter_normaliza_sku_informado TestCadastroEConsulta	Verificar que a consulta aceita o SKU em minúsculas.	Partição de equivalência (classe válida)	RF-E02, RN-E01	1	✓ Passou (1/1)
UT-E43	test_obter_com_sku_invalido_lanca_dados_invalidos TestCadastroEConsulta	Verificar que uma consulta com SKU fora do padrão é rejeitada antes da busca.	Partição de equivalência (classe inválida)	RN-E01	1	✓ Passou (1/1)
UT-E44	test_registrar_entrada_incrementa_saldo TestEntrada	Verificar que registrar entrada aumenta o saldo do produto.	Partição de equivalência (classe válida)	RF-E03	1	✓ Passou (1/1)
UT-E45	test_registrar_entrada_com_quantidade_invalida_nao_altera_saldo TestEntrada	Garantir que entradas com quantidade inválida são rejeitadas sem alterar o saldo.	Análise de valor limite (0) e partição de equivalência (tipos inválidos)	RF-E03, RN-E04	4	✓ Passou (4/4)
UT-E46	test_registrar_entrada_em_produto_inexistente TestEntrada	Verificar que não é possível registrar entrada para produto inexistente.	Partição de equivalência (classe inválida)	RF-E03	1	✓ Passou (1/1)
UT-E47	test_baixa_retorna_nome_preco_e_saldo_restante TestBaixa	Verificar que a baixa em lote atualiza os saldos e retorna nome, preço vigente e saldo restante.	Partição de equivalência (classe válida) e verificação de contrato	RF-E04, RN-P02	1	✓ Passou (1/1)
UT-E48	test_baixa_de_todo_o_saldo_gera_alerta_de_reposicao TestBaixa	Garantir que baixar exatamente o saldo zera o estoque e o produto passa a precisar de reposição.	Análise de valor limite (quantidade = saldo)	RF-E04, RN-E05	1	✓ Passou (1/1)
UT-E49	test_baixa_e_atomica_quando_um_item_nao_tem_saldo TestBaixa	Garantir que, se um item do lote não tem saldo, nenhum item é baixado (tudo ou nada).	Teste de transação/atOMICidade com valor limite (saldo+1)	RN-E06, RN-P04	1	✓ Passou (1/1)
UT-E50	test_baixa_e_atomica_quando_um_produto_nao_existe TestBaixa	Garantir que um SKU inexistente no lote impede a baixa dos demais itens.	Teste de atOMICidade (classe inválida)	RN-E06	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E51	test_baixa_considera_soma_de_itens_repetidos TestBaixa	Garantir que itens repetidos são somados antes da verificação de saldo (3 + 3 > 5).	Análise de valor limite após agrupamento	RN-E06	1	✓ Passou (1/1)
UT-E52	test_baixas_concorren-tes_nunca_deixam_saldo_negativo TestBaixa	Verificar que 30 baixas simultâneas de 1 unidade sobre saldo 10 resultam em exatamente 10 sucessos.	Teste de concorrência (condição de corrida)	RNF04	1	✓ Passou (1/1)
UT-E53	test_devolucao_incrementa_saldos TestDevolucao	Verificar que a devolução (estorno) soma as quantidades aos saldos.	Partição de equivalência (classe válida)	RF-E05	1	✓ Passou (1/1)
UT-E54	test_devolucao_com_produto_inexistente_nao_altera_nada TestDevolucao	Garantir que a devolução é atômica quando um SKU não existe.	Teste de atomicidade (classe inválida)	RF-E05, RN-E06	1	✓ Passou (1/1)
UT-E55	test_devolucao_acima_da_capacidade_nao_altera_nada TestDevolucao	Garantir que, se um item exceder a capacidade máxima, nenhum saldo é alterado.	Análise de valor limite (capacidade+1) e atomicidade	RF-E05, RN-E04	1	✓ Passou (1/1)
UT-E56	test_lista_apenas_produtos_com_saldo_ate_o_minimo TestReposicao	Verificar que a lista de reposição contém somente produtos com saldo menor ou igual ao mínimo.	Análise de valor limite (saldo = mínimo)	RF-E06, RN-E05	1	✓ Passou (1/1)

tests/unit/estoque/test_api.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E57	test_health_informacao_servico_ativo TestSaude	Verificar que o endpoint de saúde responde 200 com status ok.	Teste de rota (caixa-preta)	RNF01	1	✓ Passou (1/1)
UT-E58	test_post_produto_valido_retorna_201_com_produto TestProdutosApi	Verificar que o cadastro via API retorna 201 e o produto normalizado.	Partição de equivalência (classe válida) e contrato HTTP	RF-E01, RNF01	1	✓ Passou (1/1)
UT-E59	test_post_produto_invalido_retorna_400_padronizado TestProdutosApi	Verificar que dados inválidos retornam 400 com corpo de erro padronizado (erro + código).	Partição de equivalência (classe inválida) e contrato de erro	RN-E03, RNF01	1	✓ Passou (1/1)
UT-E60	test_post_produto_duplicado_retorna_409 TestProdutosApi	Verificar que cadastrar SKU já existente retorna 409 Conflict.	Partição de equivalência (classe inválida: duplicidade)	RN-E01	1	✓ Passou (1/1)
UT-E61	test_corpo_que_nao_e_objeto_json_retorna_400 TestProdutosApi	Garantir que corpos não-JSON ou JSON que não é objeto são rejeitados com 400.	Partição de equivalência (entrada malformada)	RNF01	2	✓ Passou (2/2)
UT-E62	test_get_lista_e_consulta_por_sku TestProdutosApi	Verificar a listagem de produtos e a consulta individual por SKU (em minúsculas).	Teste de rota (caixa-preta)	RF-E02	1	✓ Passou (1/1)
UT-E63	test_get_produto_inexistente_retorna_404 TestProdutosApi	Verificar que consultar um SKU inexistente retorna 404 com código PRODUTO_NAO_ENCONTRADO.	Partição de equivalência (classe inválida)	RF-E02, RNF01	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E64	test_post_entrada_atualiza_saldo TestProdutosApi	Verificar que a entrada via API retorna o produto com saldo atualizado.	Partição de equivalência (classe válida)	RF-E03	1	✓ Passou (1/1)
UT-E65	test_post_entrada_com_quantidade_zero_retorna_400 TestProdutosApi	Verificar que uma entrada com quantidade 0 é rejeitada com 400.	Análise de valor limite (mínimo-1)	RF-E03, RN-E04	1	✓ Passou (1/1)
UT-E66	test_get_reposicao_lista_produtos_no_minimo TestProdutosApi	Verificar que o endpoint de reposição lista os produtos cujo saldo atingiu o mínimo.	Análise de valor limite (saldo = mínimo)	RF-E06, RN-E05	1	✓ Passou (1/1)
UT-E67	test_post_baixa_retorna_itens_com_preco TestBaixaEDevolucaoApi	Verificar o contrato da baixa em lote (itens com nome, preço, quantidade e saldo).	Verificação de contrato de integração (provedor)	RF-E04, RN-P02	1	✓ Passou (1/1)
UT-E68	test_post_baixa_sem_saldo_retorna_409_com_detalhes TestBaixaEDevolucaoApi	Verificar que falta de saldo retorna 409 com sku, disponível e solicitado no corpo.	Análise de valor limite (saldo+1) e contrato de erro	RN-E04, RNF01	1	✓ Passou (1/1)
UT-E69	test_post_baixa_de_produto_inexistente_retorna_404 TestBaixaEDevolucaoApi	Verificar que a baixa de SKU inexistente retorna 404.	Partição de equivalência (classe inválida)	RF-E04	1	✓ Passou (1/1)
UT-E70	test_post_devolucao_retorna_produtos_atualizados TestBaixaEDevolucaoApi	Verificar que a devolução via API retorna os produtos com saldo atualizado.	Partição de equivalência (classe válida)	RF-E05	1	✓ Passou (1/1)
UT-E71	test_rota_inexistente_retorna_404_em_json TestErrosGenericos	Verificar que rotas desconhecidas retornam 404 no formato JSON padronizado.	Partição de equivalência (classe inválida)	RNF01	1	✓ Passou (1/1)
UT-E72	test_metodo_nao_permitido_retorna_405_em_json TestErrosGenericos	Verificar que um método HTTP não suportado retorna 405 no formato JSON padronizado.	Partição de equivalência (classe inválida)	RNF01	1	✓ Passou (1/1)
UT-E73	test_erro_de_negocio_nao_mapeado_retorna_400 TestErrosGenericos	Garantir que um erro de negócio genérico (sem status específico) é convertido em 400.	Dublê de teste (stub do serviço) para forçar o caminho de erro	RNF01	1	✓ Passou (1/1)
UT-E74	test_main_usa_host_e_porta_das_variaveis_de_ambiente TestInicializacao	Verificar que o servidor é iniciado com host e porta definidos por variáveis de ambiente.	Dublê de teste (mock de Flask.run) e configuração por ambiente	RNF05	1	✓ Passou (1/1)
UT-E75	test_execucao_como_modo_usa_porta_padrao_5001 TestInicializacao	Verificar que "python -m estoque_service" sobe o servidor na porta padrão 5001.	Dublê de teste (mock de Flask.run)	RNF05	1	✓ Passou (1/1)

tests/unit/estoque/test_cli.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E76	test_cadastrar_e_listar_produto TestMenu	Verificar que o usuário cadastra um produto pelo menu (preço com vírgula) e o vê na listagem formatada.	Teste de caixa-preta da CLI com fake HTTP (Flask em memória)	RF-E07, RF-E01, RF-E02	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-E77	test_erro_da_api_e_exibido_ao_usuario TestMenu	Verificar que um erro de validação da API (preço negativo) é mostrado com o status HTTP e a mensagem.	Partição de equivalência (classe inválida) via CLI	RF-E07, RN-E03	1	✓ Passou (1/1)
UT-E78	test_quantidade_nao_numerica_nao_chama_a_api TestMenu	Garantir que uma quantidade não numérica é recusada pela CLI antes de qualquer chamada HTTP.	Partição de equivalência (tipo inválido) e duplê espião	RF-E07	1	✓ Passou (1/1)
UT-E79	test_registrar_entrada TestMenu	Verificar a entrada de mercadoria pelo menu e a mensagem com o novo saldo.	Teste de caixa-preta da CLI	RF-E07, RF-E03	1	✓ Passou (1/1)
UT-E80	test_registrar_entrada_invalida TestMenu	Verificar as mensagens para quantidade não numérica e para produto inexistente na entrada.	Partição de equivalência (classes inválidas)	RF-E07, RF-E03	2	✓ Passou (2/2)
UT-E81	test_reposicao_lista_produtos_no_minimo TestMenu	Verificar que a opção de reposição mostra "Nenhum produto encontrado." e, com saldo no mínimo, a situação REPOR.	Análise de valor limite (saldo = mínimo) via CLI	RF-E07, RF-E06, RN-E05	1	✓ Passou (1/1)
UT-E82	test_opcao_invalida_e_fim_de_entrada TestMenu	Verificar que uma opção inexistente é avisada e que o fim da entrada (Ctrl+D/Ctrl+Z) encerra o programa.	Partição de equivalência (opção inválida) e valor limite (fim da entrada)	RF-E07	1	✓ Passou (1/1)
UT-E83	test_servidor_fora_do_ar TestMenu	Verificar que, sem o servidor do estoque, a CLI informa a falha de conexão em vez de encerrar com erro.	Injeção de falha com duplê (mock que lança ConnectionError)	RF-E07	1	✓ Passou (1/1)
UT-E84	test_moeda_no_formato_brasileiro TestFormatacaoEInicializacao	Verificar a formatação de valores em reais (separador de milhar e vírgula decimal).	Partição de equivalência e valores limite de formatação	RF-E07	3	✓ Passou (3/3)
UT-E85	test_execucao_como_modulo_usa_url_do_ambiente TestFormatacaoEInicializacao	Verificar que "python -m estoque_service.cli" usa a URL de ESTOQUE_URL e encerra ao fim da entrada.	Configuração por ambiente e duplê de input	RF-E07, RNF05	1	✓ Passou (1/1)

5.3 Testes unitários: Aplicação 2: Pedidos

tests/unit/pedidos/test dominio.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P01	test_desconto_por_faixa_de_subtotal TestDescontoProgressivo	Verificar a taxa e o valor do desconto nas fronteiras das faixas (499,99 / 500,00 / 999,99 / 1000,00) com arredondamento half-up.	Análise de valor limite e partição de equivalência (3 faixas)	RN-P03	6	✓ Passou (6/6)
UT-P02	test_normaliza_espacos_internos_e_das_pontas TestValidarCliente	Verificar que espaços repetidos e das extremidades são removidos do nome do cliente.	Partição de equivalência (classe válida)	RN-P01	1	✓ Passou (1/1)
UT-P03	test_aceita_nomes_nos_limites TestValidarCliente	Garantir que nomes com 3 e 80 caracteres (limites) são aceitos.	Análise de valor limite (mínimo e máximo)	RN-P01	2	✓ Passou (2/2)
UT-P04	test_rejeita_nomes_fora_dos_limites TestValidarCliente	Garantir que nomes com 2 e 81 caracteres são rejeitados.	Análise de valor limite (mínimo-1 e máximo+1)	RN-P01	2	✓ Passou (2/2)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P05	test_rejeita_cliente_ausente_ou_invalido TestValidarCliente	Verificar que cliente ausente, vazio ou não textual é rejeitado.	Partição de equivalência (classes inválidas)	RN-P01	4	✓ Passou (4/4)
UT-P06	test_validar_sku_normaliza_e_rejeita_invalidos TestValidarItens	Verificar que o SKU é normalizado para maiúsculas e que formatos inválidos são rejeitados.	Partição de equivalência (válida e inválidas)	RN-P01	1	✓ Passou (1/1)
UT-P07	test_quantidade_por_item_entre_1_e_100 TestValidarItens	Verificar os limites da quantidade por item (0 e 101 inválidos; 1 e 100 válidos).	Análise de valor limite	RN-P01	4	✓ Passou (4/4)
UT-P08	test_quantidade_deve_ser_inteira TestValidarItens	Verificar que booleanos, floats, textos e ausência não são aceitos como quantidade.	Partição de equivalência (tipos inválidos)	RN-P01	4	✓ Passou (4/4)
UT-P09	test_agrupa_itens_repetidos_mantendo_ordem TestValidarItens	Verificar que itens com o mesmo SKU são somados, preservando a ordem da primeira ocorrência.	Partição de equivalência (classe válida com repetição)	RN-P01	1	✓ Passou (1/1)
UT-P10	test_rejeita_soma_de_itens_repetidos_acima_de_100 TestValidarItens	Garantir que o limite de 100 unidades vale para a soma dos itens repetidos (60 + 41).	Análise de valor limite após agrupamento	RN-P01	1	✓ Passou (1/1)
UT-P11	test_aceita_ate_20_produtos_distintos TestValidarItens	Garantir que um pedido com exatamente 20 produtos distintos é aceito.	Análise de valor limite (máximo)	RN-P01	1	✓ Passou (1/1)
UT-P12	test_rejeita_21_produtos_distintos TestValidarItens	Garantir que um pedido com 21 produtos distintos é rejeitado.	Análise de valor limite (máximo+1)	RN-P01	1	✓ Passou (1/1)
UT-P13	test_rejeita_pedido_sem_itens TestValidarItens	Verificar que o pedido precisa de uma lista com pelo menos um item.	Análise de valor limite (0 itens) e partição de equivalência	RN-P01	4	✓ Passou (4/4)
UT-P14	test_rejeita_item_que_nao_e_objeto TestValidarItens	Verificar que cada item deve ser um objeto com sku e quantidade.	Partição de equivalência (tipo inválido)	RN-P01, RNF01	1	✓ Passou (1/1)
UT-P15	test_validar_solicitacao_retorna_cliente_e_itens TestValidarItens	Verificar que uma solicitação válida resulta no par (cliente, itens normalizados).	Partição de equivalência (classe válida)	RF-P02, RN-P01	1	✓ Passou (1/1)
UT-P16	test_validar_solicitacao_rejeita_corpo_que_nao_e_objeto TestValidarItens	Verificar que a solicitação precisa ser um objeto JSON.	Partição de equivalência (tipo inválido)	RF-P02, RNF01	3	✓ Passou (3/3)
UT-P17	test_item_calcula_total_e_serializa TestPedido	Verificar o total do item (preço × quantidade) e seu formato JSON.	Verificação de cálculo e contrato de saída	RF-P02, RNF01	1	✓ Passou (1/1)
UT-P18	test_totais_sem_desconto TestPedido	Verificar subtotal, desconto zero e total para pedido abaixo de R\$ 500,00.	Partição de equivalência (faixa sem desconto)	RN-P03	1	✓ Passou (1/1)
UT-P19	test_totais_com_desconto_de_5_por_cento TestPedido	Verificar o desconto de 5% quando o subtotal atinge exatamente R\$ 500,00.	Análise de valor limite (início da faixa de 5%)	RN-P03	1	✓ Passou (1/1)
UT-P20	test_totais_com_desconto_de_10_por_cento TestPedido	Verificar o desconto de 10% para subtotal acima de R\$ 1.000,00.	Partição de equivalência (faixa de 10%)	RN-P03	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P21	test_pedido_novo_nasc_e_confirmado_e_serializa TestPedido	Verificar que o pedido nasce CONFIRMADO e o formato JSON completo (incluindo percentual de desconto).	Verificação de estado inicial e contrato de saída	RF-P02, RF-P03, RNF01	1	✓ Passou (1/1)
UT-P22	test_cancelar_pedido_confirmado_registra_data TestPedido	Verificar a transição CONFIRMADO → CANCELADO e o registro da data de cancelamento.	Teste de transição de estados	RF-P04, RN-P05	1	✓ Passou (1/1)
UT-P23	test_cancelar_sem_momento_usa_relogio_atual TestPedido	Verificar que, sem momento informado, o cancelamento usa a data/hora atual (UTC).	Partição de equivalência (parâmetro opcional omitido)	RF-P04	1	✓ Passou (1/1)
UT-P24	test_cancelar_pedido_ja_cancelado_e_transicao_invalida TestPedido	Garantir que um pedido CANCELADO não pode ser cancelado novamente.	Teste de transição de estados (transição inválida)	RN-P05	1	✓ Passou (1/1)
UT-P25	test_pedido_nao_encontrado Informa id TestPedido	Verificar que a exceção de pedido inexistente expõe o id e o código padronizado.	Verificação de contrato de erro	RF-P03, RNF01	1	✓ Passou (1/1)

tests/unit/pedidos/test_cliente_estoque.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P26	test_remove_barra_final_e_cria_sessao_padrao TestConfiguracao	Verificar que a URL base é normalizada e que uma sessão requests é criada quando não injetada.	Verificação de configuração padrão	RNF03, RNF05	1	✓ Passou (1/1)
UT-P27	test_listar_produtos_faz_get_com_timeout TestChamadasBemSucedidas	Verificar que o catálogo é obtido via GET /api/produtos respeitando o timeout configurado.	Dublê de teste (mock) para verificar a interação HTTP	RF-P01, RNF03	1	✓ Passou (1/1)
UT-P28	test_baixar_envia_itens_e_converte_preco_para_decimal TestChamadasBemSucedidas	Verificar que a baixa envia POST com os itens e converte o preço recebido para Decimal.	Dublê de teste (mock) e verificação de contrato de integração (consumidor)	RF-E04, RN-P02	1	✓ Passou (1/1)
UT-P29	test_devolver_envia_post_de_devolucao TestChamadasBemSucedidas	Verificar que a devolução envia POST /api/estoque/devolucoes com os itens.	Dublê de teste (mock) para verificar a interação HTTP	RF-E05, RF-P04	1	✓ Passou (1/1)
UT-P30	test_esta_disponivel_quando_health_responde TestChamadasBemSucedidas	Verificar que o estoque é considerado disponível quando GET /health responde 200.	Dublê de teste (stub)	RF-P05	1	✓ Passou (1/1)
UT-P31	test_erros_de_negocio_do_estoque_viram_excecoes_do_pedido TestTraducaoDeErros	Verificar que cada código de erro do estoque é traduzido para a exceção correspondente, preservando a mensagem.	Tabela de decisão (código HTTP/código de erro → exceção)	RN-P04, RNF01	3	✓ Passou (3/3)
UT-P32	test_erro_4xx_desconhecido_vira_resposta_invalida TestTraducaoDeErros	Garantir que erros 4xx sem código conhecido (ou com corpo inesperado) viram RespostaInvalidaEstoque.	Tabela de decisão (caso padrão) e partição de equivalência	RN-P06	2	✓ Passou (2/2)
UT-P33	test_erro_5xx_indica_estoque_indisponivel TestTraducaoDeErros	Verificar que respostas 5xx do estoque resultam em EstoqueIndisponivel.	Partição de equivalência (classe 5xx)	RN-P06	3	✓ Passou (3/3)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P34	test_timeout_indica_estoque_indisponivel TestTraducaoDeErros	Verificar que um timeout na chamada HTTP resulta em EstoqueIndisponivel com mensagem de tempo limite.	Dublê de teste (mock que lança exceção) - injeção de falha	RN-P06, RNF03	1	✓ Passou (1/1)
UT-P35	test_falha_de_conexao_indica_estoque_indisponivel TestTraducaoDeErros	Verificar que uma falha de conexão resulta em EstoqueIndisponivel.	Dublê de teste (mock que lança exceção) - injeção de falha	RN-P06	1	✓ Passou (1/1)
UT-P36	test_resposta_que_nao_e_json_e_invalida TestTraducaoDeErros	Garantir que uma resposta 200 sem JSON válido é tratada como RespostaInvalidaEstoque.	Dublê de teste (stub) com resposta malformada	RN-P06	1	✓ Passou (1/1)
UT-P37	test_catalogo_em_formato_inesperado_e_invalido TestTraducaoDeErros	Garantir que um catálogo que não é uma lista é rejeitado.	Dublê de teste (stub) com contrato violado	RF-P01, RN-P06	1	✓ Passou (1/1)
UT-P38	test_resposta_de_baixa_com_contrato_violado_e_invalida TestTraducaoDeErros	Garantir que respostas de baixa sem campos obrigatórios ou com valores inválidos são rejeitadas.	Partição de equivalência (violações do contrato de integração)	RN-P02, RN-P06	5	✓ Passou (5/5)
UT-P39	test_esta_disponivel_re torna_falso_quando_estoque_falha TestTraducaoDeErros	Verificar que falhas de comunicação fazem esta_disponivel() retornar False, sem lançar exceção.	Dublê de teste (mock que lança exceção)	RF-P05	1	✓ Passou (1/1)

tests/unit/pedidos/test_servico_e_repositorio.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P40	test_ids_sao_sequenciais_e_listagem_e_do_mais_recente TestRepositorioPedidos	Verificar que o repositório gera ids sequenciais e lista do pedido mais recente para o mais antigo.	Verificação de estado	RF-P03	1	✓ Passou (1/1)
UT-P41	test_obter_id_inexistente_lanca_erro TestRepositorioPedidos	Verificar que consultar um id inexistente lança PedidoNaoEncontrado.	Partição de equivalência (classe inválida)	RF-P03	1	✓ Passou (1/1)
UT-P42	test_cria_pedido_confirmado_com_preco_vindo_do_estoque TestCriarPedido	Verificar que o pedido é criado CONFIRMADO, com preços informados pelo estoque (ignorando preço enviado pelo cliente) e data do relógio injetado.	Dublê de teste (fake do estoque) e injeção de dependência (relógio)	RF-P02, RN-P02	1	✓ Passou (1/1)
UT-P43	test_envia_ao_estoque_itens_normalizados_e_agrupados TestCriarPedido	Verificar que o estoque recebe os itens já validados, normalizados e agrupados.	Dublê de teste (spy) para verificar a interação	RN-P01, RN-P04	1	✓ Passou (1/1)
UT-P44	test_dados_invalidos_nao_chegam_ao_estoque TestCriarPedido	Garantir que uma solicitação inválida é rejeitada antes de qualquer chamada ao estoque.	Dublê de teste (mock) com verificação de ausência de chamada	RN-P01	1	✓ Passou (1/1)
UT-P45	test_recusa_do_estoque_nao_gera_pedido TestCriarPedido	Garantir que, se o estoque recusar a reserva (saldo insuficiente ou produto inexistente), nenhum pedido é registrado.	Dublê de teste (fake) com injeção de falha	RN-P04	2	✓ Passou (2/2)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P46	test_estoque_indisponivel_nao_gera_pedido TestCriarPedido	Garantir que, com o estoque fora do ar, a criação falha com EstoqueIndisponivel e nada é registrado.	Dublê de teste (fake) com injeção de falha	RN-P06	1	✓ Passou (1/1)
UT-P47	test_listar_e_obter_pedidos TestConsultas	Verificar a listagem (mais recente primeiro) e a consulta de pedidos por id.	Verificação de estado	RF-P03	1	✓ Passou (1/1)
UT-P48	test_obter_pedido_inexistente TestConsultas	Verificar que consultar um pedido inexistente lança PedidoNaoEncontrado.	Partição de equivalência (classe inválida)	RF-P03	1	✓ Passou (1/1)
UT-P49	test_catalogo_e_saude_delegam_ao_estoque TestConsultas	Verificar que catálogo e verificação de saúde são obtidos do cliente de estoque injetado.	Dublê de teste (fake)	RF-P01, RF-P05	1	✓ Passou (1/1)
UT-P50	test_cancelamento_devolve_itens_e_marca_cancelado TestCancelarPedido	Verificar que cancelar devolve os itens ao estoque e marca o pedido como CANCELADO com a data do relógio.	Dublê de teste (spy) e teste de transição de estados	RF-P04, RN-P05, RF-E05	1	✓ Passou (1/1)
UT-P51	test_cancelar_duas vezes_nao_devolve_em_dobro TestCancelarPedido	Garantir que o segundo cancelamento é recusado e o estoque recebe a devolução uma única vez.	Teste de transição de estados (transição inválida)	RN-P05	1	✓ Passou (1/1)
UT-P52	test_estoque_indisponivel_mantem_pedido_confirmado TestCancelarPedido	Garantir que, se o estoque estiver fora do ar no cancelamento, o pedido permanece CONFIRMADO (consistência entre serviços).	Dublê de teste (fake) com injeção de falha	RN-P05, RN-P06	1	✓ Passou (1/1)
UT-P53	test_cancelar_pedido_inexistente_nao_chama_estoque TestCancelarPedido	Verificar que cancelar um id inexistente lança PedidoNaoEncontrado sem chamar o estoque.	Dublê de teste (mock) com verificação de ausência de chamada	RF-P04	1	✓ Passou (1/1)

tests/unit/pedidos/test_api.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P54	test_health_informacao_da_dependencia_estoque TestSaude	Verificar que o health check do serviço de pedidos informa se o estoque está disponível.	Partição de equivalência (estoque online/offline) com dublê	RF-P05	2	✓ Passou (2/2)
UT-P55	test_catalogo_retorna_produtos_do_estoque TestCatalogoApi	Verificar que o catálogo exposto pelo serviço de pedidos vem do estoque.	Dublê de teste (fake)	RF-P01	1	✓ Passou (1/1)
UT-P56	test_catalogo_com_estoque_fora_do_ar_retorna_503 TestCatalogoApi	Verificar que, com o estoque indisponível, o catálogo responde 503 com código ESTOQUE_INDISPONIVEL.	Injeção de falha com dublê	RF-P01, RN-P06	1	✓ Passou (1/1)
UT-P57	test_post_pedido_valido_retorna_201_com_totais TestCriarPedidoApi	Verificar que um pedido válido retorna 201 com status, itens, desconto e total.	Partição de equivalência (classe válida) e contrato HTTP	RF-P02, RN-P03, RNF01	1	✓ Passou (1/1)
UT-P58	test_post_pedido_invalido_retorna_400 TestCriarPedidoApi	Verificar que pedido sem itens, sem cliente ou sem corpo JSON retorna 400.	Partição de equivalência (classes inválidas)	RN-P01, RNF01	3	✓ Passou (3/3)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P59	test_recusas_do_estoque_v iram_status_http_adequado s TestCriarPedidoApi	Verificar que saldo insuficiente gera 409 e produto inexistente gera 422.	Tabela de decisão (erro → status HTTP)	RN-P04, RNF01	2	✓ Passou (2/2)
UT-P60	test_estoque_indisponivel _retorna_503 TestCriarPedidoApi	Verificar que, com o estoque fora do ar, a criação de pedido responde 503.	Injeção de falha com duplê	RN-P06	1	✓ Passou (1/1)
UT-P61	test_resposta_invalida_do _estoque_retorna_502 TestCriarPedidoApi	Verificar que uma resposta fora do contrato vinda do estoque gera 502 Bad Gateway.	Dublê de teste (stub que lança exceção)	RN-P06	1	✓ Passou (1/1)
UT-P62	test_listar_e_consultar_p edido TestConsultaECancelamentoApi	Verificar a listagem de pedidos e a consulta por id via API.	Teste de rota (caixa-preta)	RF-P03	1	✓ Passou (1/1)
UT-P63	test_consultar_pedido_ine xistente_retorna_404 TestConsultaECancelamentoApi	Verificar que consultar um id inexistente retorna 404 com código PEDIDO_NAO_ENCONTRADO.	Partição de equivalência (classe inválida)	RF-P03, RNF01	1	✓ Passou (1/1)
UT-P64	test_cancelar_pedido_reto rna_200_e_segunda_vez_409 TestConsultaECancelamentoApi	Verificar que o primeiro cancelamento retorna 200 (CANCELADO) e o segundo 409 (transição inválida).	Teste de transição de estados via API	RF-P04, RN-P05	1	✓ Passou (1/1)
UT-P65	test_rota_inexistente_e_m etodo_nao_permitido_em_js on TestErrosGenericos	Verificar que rota desconhecida (404) e método não suportado (405) retornam JSON padronizado.	Partição de equivalência (classes inválidas)	RNF01	1	✓ Passou (1/1)
UT-P66	test_erro_de_negocio_nao_ mapeado_retorna_400 TestErrosGenericos	Garantir que um erro de negócio genérico (sem status específico) é convertido em 400.	Dublê de teste (stub do serviço) para forçar o caminho de erro	RNF01	1	✓ Passou (1/1)
UT-P67	test_servico_padrao_le_ur l_e_timeout_do_ambiente TestConfiguracaoEInicializacao	Verificar que a URL e o timeout do estoque são lidos das variáveis ESTOQUE_URL e ESTOQUE_TIMEOUT.	Configuração por ambiente (monkeypatch)	RNF03, RNF05	1	✓ Passou (1/1)
UT-P68	test_app_sem_servico_inje tado_usa_configuracao_pad rao TestConfiguracaoEInicializacao	Verificar que, sem variáveis de ambiente, o app aponta para o estoque em http://127.0.0.1:5001 com timeout de 3 s.	Verificação de configuração padrão	RNF03, RNF05	1	✓ Passou (1/1)
UT-P69	test_main_usa_host_e_port a_das_variaveis_de_ambien te TestConfiguracaoEInicializacao	Verificar que o servidor é iniciado com host e porta definidos por variáveis de ambiente.	Dublê de teste (mock de Flask.run) e configuração por ambiente	RNF05	1	✓ Passou (1/1)
UT-P70	test_execucao_como_modulo _usa_porta_padrao_5002 TestConfiguracaoEInicializacao	Verificar que "python -m pedidos_service" sobe o servidor na porta padrão 5002.	Dublê de teste (mock de Flask.run)	RNF05	1	✓ Passou (1/1)

tests/unit/pedidos/test_cli.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P71	test_catalogo_mostra_pr odutos_do_estoque TestCatalogo	Verificar que a opção de catálogo lista os produtos vindos do Estoque com preço e quantidade disponível.	Teste de caixa-preta da CLI com fakes (Flask em memória + estoque falso)	RF-P06, RF-P01	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
UT-P72	test_catalogo_vazio TestCatalogo	Verificar a mensagem exibida quando o Estoque não tem produtos cadastrados.	Análise de valor limite (lista vazia)	RF-P06, RF-P01	1	✓ Passou (1/1)
UT-P73	test_catalogo_com_estoque_fora_do_ar TestCatalogo	Verificar que a indisponibilidade do Estoque chega ao usuário como erro HTTP 503 com mensagem clara.	Injeção de falha com dublê	RF-P06, RN-P06	1	✓ Passou (1/1)
UT-P74	test_criar_pedido_mostrar_itens_desconto_e_total TestPedidos	Verificar que o pedido digitado item a item é confirmado e exibido com itens, desconto de 5% e total.	Teste de caixa-preta da CLI (fluxo principal)	RF-P06, RF-P02, RN-P03	1	✓ Passou (1/1)
UT-P75	test_quantidade_invalida_ignora_item TestPedidos	Verificar que um item com quantidade não numérica é ignorado e que um pedido sem itens é recusado pela API (400).	Partição de equivalência (classes inválidas)	RF-P06, RN-P01	1	✓ Passou (1/1)
UT-P76	test_pedido_sem_saldo_e_recusado TestPedidos	Verificar que um pedido acima do saldo é recusado com HTTP 409 e o saldo do estoque não muda.	Análise de valor limite (saldo+1) via CLI	RF-P06, RN-P04	1	✓ Passou (1/1)
UT-P77	test_listar_pedidos TestPedidos	Verificar a listagem vazia e, depois de um pedido abaixo de R\$ 500,00, a indicação "sem desconto".	Análise de valor limite (lista vazia) e partição (faixa sem desconto)	RF-P06, RF-P03, RN-P03	1	✓ Passou (1/1)
UT-P78	test_cancelar_pedido TestPedidos	Verificar o cancelamento pelo menu, a devolução ao estoque e a recusa de um segundo cancelamento (409).	Teste de transição de estados via CLI	RF-P06, RF-P04, RN-P05	1	✓ Passou (1/1)
UT-P79	test_cancelar_numero_invalido_ou_inexistente TestPedidos	Verificar as mensagens para número de pedido não numérico e para pedido inexistente.	Partição de equivalência (classes inválidas)	RF-P06, RF-P04	2	✓ Passou (2/2)
UT-P80	test_situacao_dos_servicos TestSituacaoEFalhas	Verificar que a opção de situação mostra o estado do Pedidos e do Estoque (online/offline).	Partição de equivalência (estoque disponível/indisponível)	RF-P06, RF-P05	2	✓ Passou (2/2)
UT-P81	test_opcao_invalida_e_fim_de_entrada TestSituacaoEFalhas	Verificar que uma opção inexistente é avisada e que o fim da entrada encerra o programa.	Partição de equivalência (opção inválida) e valor limite (fim da entrada)	RF-P06	1	✓ Passou (1/1)
UT-P82	test_servidor_de_pedido_s_fora_do_ar TestSituacaoEFalhas	Verificar que, sem o servidor de pedidos, cada opção informa a falha de conexão sem encerrar a CLI.	Injeção de falha com dublê (mock que lança ConnectionError)	RF-P06	1	✓ Passou (1/1)
UT-P83	test_moeda_no_formato_brasileiro TestFormatacaoEInicializacao	Verificar a formatação de valores em reais usada nos resumos de pedido.	Partição de equivalência	RF-P06	1	✓ Passou (1/1)
UT-P84	test_execucao_como_modulo_usa_url_do_ambiente TestFormatacaoEInicializacao	Verificar que "python -m pedidos_service.cli" usa a URL de PEDIDOS_URL e encerra ao fim da entrada.	Configuração por ambiente e dublê de input	RF-P06, RNF05	1	✓ Passou (1/1)

5.4 Testes de integração: Estoque + Pedidos

tests/integration/test_integracao_estoque_pedidos.py

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
IT01	test_pedidos_reconhece_estoque_online TestComunicacaoEntreServicos	Verificar que os dois serviços sobem em processos distintos e que o health check do Pedidos confirma a comunicação com o Estoque.	Teste de integração (smoke test entre serviços)	RF-P05, RNF05	1	✓ Passou (1/1)
IT02	test_catalogo_do_pedidos_reflete_produtos_do_estoque TestComunicacaoEntreServicos	Verificar que um produto cadastrado no Estoque aparece imediatamente no catálogo do Pedidos, com preço e saldo.	Teste de integração (consistência de dados entre serviços)	RF-P01, RF-E01	1	✓ Passou (1/1)
IT03	test_pedido_confirmado_baixa_saldo_no_estoque TestCriacaoDePedido	Verificar que um pedido confirmado no Pedidos reduz o saldo correspondente no Estoque.	Teste de integração (fluxo principal / caminho feliz)	RF-P02, RF-E04	1	✓ Passou (1/1)
IT04	test_preco_do_pedido_vem_do_estoque TestCriacaoDePedido	Garantir que o preço unitário usado no pedido é o do Estoque, ignorando um preço enviado pelo cliente.	Teste de integração (fonte da verdade entre serviços)	RN-P02	1	✓ Passou (1/1)
IT05	test_desconto_progressivo_com_precos_reais_do_estoque TestCriacaoDePedido	Verificar o desconto de 0%, 5% e 10% nas fronteiras das faixas usando preços cadastrados no Estoque.	Teste de integração com análise de valor limite	RN-P03, RN-P02	3	✓ Passou (3/3)
IT06	test_estoque_insuficiente_recusa_pedido_sem_baixa_parcial TestCriacaoDePedido	Garantir que, se um dos itens não tem saldo, o pedido é recusado (409) e nenhum item é baixado no Estoque.	Teste de integração (atomicidade entre serviços) com valor limite (saldo+1)	RN-P04, RN-E06	1	✓ Passou (1/1)
IT07	test_produto_inexistente_retorna_422_sem_alterar_estoque TestCriacaoDePedido	Verificar que um pedido com SKU que não existe no Estoque retorna 422 e não altera os demais saldos.	Teste de integração (propagação de erro entre serviços)	RN-P04, RNF01	1	✓ Passou (1/1)
IT08	test_pedido_invalido_nao_altera_estoque TestCriacaoDePedido	Garantir que um pedido rejeitado na validação do Pedidos (quantidade 101) não gera nenhuma baixa no Estoque.	Teste de integração com valor limite (máximo+1)	RN-P01	1	✓ Passou (1/1)
IT09	test_entrada_no_estoque_libera_pedido_antes_recusado TestCriacaoDePedido	Verificar que, após registrar entrada no Estoque, um pedido antes recusado por falta de saldo passa a ser aceito.	Teste de integração (cenário de ponta a ponta)	RF-E03, RF-P02, RN-P04	1	✓ Passou (1/1)
IT10	test_pedido_que_atinge_minimo_gera_alerta_de_reposicao TestCriacaoDePedido	Verificar que um pedido que leva o saldo ao estoque mínimo faz o produto aparecer na lista de reposição do Estoque.	Teste de integração com valor limite (saldo = mínimo)	RN-E05, RF-E06	1	✓ Passou (1/1)
IT11	test_cancelamento_devolve_itens_ao_estoque TestCancelamento	Verificar que cancelar um pedido no Pedidos devolve as quantidades ao Estoque.	Teste de integração (fluxo de compensação)	RF-P04, RF-E05	1	✓ Passou (1/1)
IT12	test_cancelamento_duplicado_nao_devolve_em_dobro TestCancelamento	Garantir que cancelar o mesmo pedido duas vezes retorna 409 e o Estoque recebe a devolução só uma vez.	Teste de integração (transição de estado inválida)	RN-P05	1	✓ Passou (1/1)
IT13	test_pedidos_simultaneos_nao_vendem_alem_do_saldo TestConcorrenca	Verificar que 25 pedidos simultâneos de 1 unidade sobre saldo 10 resultam em exatamente 10 confirmados e 15 recusados.	Teste de integração de concorrência (condição de corrida)	RNF04, RN-P04	1	✓ Passou (1/1)

ID	Teste	Objetivo	Técnica	Requisitos	Casos	Resultado
IT14	test_cancelamentos_simultaneos_devolvem_uma_unica_voz TestConcorrenzia	Garantir que 10 cancelamentos simultâneos do mesmo pedido resultam em 1 sucesso e devolução única ao Estoque.	Teste de integração de concorrência (condição de corrida)	RNF04, RN-P05	1	✓ Passou (1/1)
IT15	test_estoque_fora_do_ar_retorna_503 TestFalhasDeComunicacao	Verificar que, sem o Estoque no ar, o Pedidos informa a indisponibilidade no health check e responde 503 ao catálogo e à criação de pedidos.	Teste de integração com injeção de falha (serviço dependente inacessível)	RN-P06, RF-P05	1	✓ Passou (1/1)
IT16	test_estoque_que_nao_responde_gera_timeout_e_503 TestFalhasDeComunicacao	Garantir que um Estoque que aceita a conexão mas não responde é abandonado após o timeout configurado (0,5 s), retornando 503.	Teste de integração com injeção de falha (latência / timeout)	RNF03, RN-P06	1	✓ Passou (1/1)
IT17	test_queda_do_estoque_mantem_pedido_confirmado TestFalhasDeComunicacao	Verificar que, se o Estoque cair depois da venda, o cancelamento responde 503 e o pedido continua CONFIRMADO (sem perda de consistência).	Teste de integração com injeção de falha (queda do processo do Estoque)	RN-P05, RN-P06	1	✓ Passou (1/1)
IT18	test_fluxo_completo_pelas_duas_clis TestInterfaceDeLinhaDeComando	Verificar o uso real pelo terminal: cadastro na CLI do Estoque, pedido na CLI do Pedidos e saldo atualizado visto novamente na CLI do Estoque.	Teste de integração ponta a ponta (CLI → API → outra API)	RF-E07, RF-P06, RF-P02, RF-E04	1	✓ Passou (1/1)
IT19	test_cli_de_pedidos_mostra_recusa_do_estoque TestInterfaceDeLinhaDeComando	Verificar que a recusa por falta de saldo, gerada no Estoque, chega ao usuário da CLI de Pedidos como HTTP 409.	Teste de integração com valor limite (saldo+1) propagado entre serviços	RF-P06, RN-P04	1	✓ Passou (1/1)

6 Evidências de execução

Todas as evidências abaixo foram produzidas automaticamente pelo script `scripts/gerar_entrega.py` em **24/09/2026 às 11:45** (Python 3.11.15, Linux 6.18.44-fc-v37). Os arquivos originais (saídas de terminal, relatórios JUnit/HTML e relatório de cobertura) acompanham a entrega na pasta `entrega/evidencias/`.

6.1 Resumo

Suíte	Comando	Casos	Aprovados	Falhas	Tempo
Unitários	<code>pytest tests/unit --cov --cov-fail-under=90</code>	256	256	0	2.06 s
Integração	<code>pytest tests/integration</code>	21	21	0	4.27 s

100%

cobertura de linhas (723/723)

100%

cobertura de ramos (148/148)

100%

cobertura combinada (critério do coverage.py)

6.2 Testes unitários e cobertura

Cobertura por arquivo das duas aplicações, obtida com `pytest-cov` (coverage.py com medição de ramos, `branch = true`):

Arquivo	Instruções	Não executadas	Ramos	Ramos parciais	Cobertura
<code>estoque_service/__init__.py</code>	2	0	0	0	100%
<code>estoque_service/__main__.py</code>	8	0	2	0	100%
<code>estoque_service/app.py</code>	57	0	4	0	100%
<code>estoque_service/cli.py</code>	92	0	26	0	100%
<code>estoque_service/dominio.py</code>	99	0	28	0	100%
<code>estoque_service/repositorio.py</code>	18	0	2	0	100%
<code>estoque_service/servico.py</code>	58	0	18	0	100%
<code>pedidos_service/__init__.py</code>	2	0	0	0	100%
<code>pedidos_service/__main__.py</code>	8	0	2	0	100%
<code>pedidos_service/app.py</code>	49	0	0	0	100%
<code>pedidos_service/cli.py</code>	102	0	32	0	100%
<code>pedidos_service/cliente_estoque.py</code>	54	0	8	0	100%
<code>pedidos_service/dominio.py</code>	118	0	26	0	100%
<code>pedidos_service/repositorio.py</code>	22	0	0	0	100%
<code>pedidos_service/servico.py</code>	34	0	0	0	100%
TOTAL	723	0	148	0	100%

Trecho final da saída do terminal (a saída completa, com cada teste, está no Apêndice A):

```
$ pytest tests/unit -v --cov --cov-report=term-missing --cov-fail-under=90
----- coverage: platform linux, python 3.11.15-final-0 -----
Name                               Stmts  Miss Branch BrPart  Cover   Missing
-----
estoque_service/__init__.py         2      0      0      0    100%
estoque_service/__main__.py         8      0      2      0    100%
estoque_service/app.py              57      0      4      0    100%
estoque_service/cli.py             92      0     26      0    100%
estoque_service/ dominio.py         99      0     28      0    100%
estoque_service/repositorio.py      18      0      2      0    100%
estoque_service/servico.py          58      0     18      0    100%
pedidos_service/__init__.py         2      0      0      0    100%
pedidos_service/__main__.py         8      0      2      0    100%
pedidos_service/app.py              49      0      0      0    100%
pedidos_service/cli.py             102     0     32      0    100%
pedidos_service/cliente_estoque.py  54      0      8      0    100%
pedidos_service/ dominio.py        118     0     26      0    100%
pedidos_service/repositorio.py      22      0      0      0    100%
pedidos_service/servico.py          34      0      0      0    100%
-----
TOTAL                               723      0    148      0    100%
Coverage HTML written to dir
/home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/cobertura_html
Coverage JSON written to file
/home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/cobertura.json

Required test coverage of 90% reached. Total coverage: 100.00%
- Generated html report:
file:///home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/relatorio-testes-unitarios.html
-
===== 256 passed in 2.05s
=====
```

Cobertura dos testes unitários: 100%

Files

Functions

Classes



☐ hide covered

coverage.py v7.16.1, created at 2026-09-24 11:45 +0000

File ▲	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	coverage
estoque_service/__init__.py	100%	2	0	0	100%	0	0	100%
estoque_service/__main__.py	100%	8	0	0	100%	2	0	100%
estoque_service/app.py	100%	57	0	0	100%	4	0	100%
estoque_service/cli.py	100%	92	0	0	100%	26	0	100%
estoque_service/dominio.py	100%	99	0	0	100%	28	0	100%
estoque_service/repositorio.py	100%	18	0	0	100%	2	0	100%
estoque_service/servico.py	100%	58	0	0	100%	18	0	100%
pedidos_service/__init__.py	100%	2	0	0	100%	0	0	100%
pedidos_service/__main__.py	100%	8	0	0	100%	2	0	100%
pedidos_service/app.py	100%	49	0	0	100%	0	0	100%
pedidos_service/cli.py	100%	102	0	0	100%	32	0	100%
pedidos_service/cliente_estoque.py	100%	54	0	0	100%	8	0	100%
pedidos_service/dominio.py	100%	118	0	0	100%	26	0	100%
pedidos_service/repositorio.py	100%	22	0	0	100%	0	0	100%
pedidos_service/servico.py	100%	34	0	0	100%	0	0	100%
Total	100%	723	0	0	100%	148	0	100%

coverage.py v7.16.1, created at 2026-09-24 11:45 +0000

Figura 2: Relatório HTML de cobertura gerado pelo coverage.py (entrega/evidencias/cobertura_html/index.html).

6.3 Testes de integração

Saída completa da execução. Cada teste utiliza as duas aplicações reais, iniciadas como processos

independentes (python -m estoque_service e python -m pedidos_service).

```
$ pytest tests/integration -v
===== test session starts =====
platform linux -- Python 3.11.15, pytest-8.3.3, pluggy-1.6.0 --
/home/user/Projeto_Teste_Requisitos_de_Software/.venv/bin/python
metadata: {'Python': '3.11.15', 'Platform': 'Linux-6.18.44-fc-v37-x86_64-with-glibc2.39', 'Packages':
{'pytest': '8.3.3', 'pluggy': '1.6.0'}, 'Plugins': {'metadata': '3.1.1', 'html': '4.1.1', 'cov': '5.0.0'},
'JAVA_HOME': '/usr/lib/jvm/java-21-openjdk-amd64'}
rootdir: /home/user/Projeto_Teste_Requisitos_de_Software
configfile: pyproject.toml
plugins: metadata-3.1.1, html-4.1.1, cov-5.0.0
collecting ... collected 21 items

tests/integration/test_integracao_estoque_pedidos.py::TestComunicacaoEntreServicos::test_pedidos_reconhece
_estoque_online PASSED [ 4%]
tests/integration/test_integracao_estoque_pedidos.py::TestComunicacaoEntreServicos::test_catalogo_do_pedid
os_reflete_produtos_do_estoque PASSED [ 9%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_pedido_confirmado_baixa_sa
ldo_no_estoque PASSED [ 14%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_preco_do_pedido_vem_do_est
oque PASSED [ 19%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_desconto_progressivo_com_p
recos_reais_do_estoque[499.99-1-0-499.99] PASSED [ 23%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_desconto_progressivo_com_p
recos_reais_do_estoque[250.00-2-5-475.00] PASSED [ 28%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_desconto_progressivo_com_p
recos_reais_do_estoque[1000.00-1-10-900.00] PASSED [ 33%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_estoque_insuficiente_recus
a_pedido_sem_baixa_parcial PASSED [ 38%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_produto_inexistente_retorn
a_422_sem_alterar_estoque PASSED [ 42%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_pedido_invalido_nao_altera
_estoque PASSED [ 47%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_entrada_no_estoque_libera
_pedido_antes_recusado PASSED [ 52%]
tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_pedido_que_atinge_minimo_g
era_alerta_de_reposicao PASSED [ 57%]
tests/integration/test_integracao_estoque_pedidos.py::TestCancelamento::test_cancelamento_devolve_itens_ao
_estoque PASSED [ 61%]
tests/integration/test_integracao_estoque_pedidos.py::TestCancelamento::test_cancelamento_duplicado_nao_de
volve_em_dobro PASSED [ 66%]
tests/integration/test_integracao_estoque_pedidos.py::TestConcorrenca::test_pedidos_simultaneos_nao_vende
m_alem_do_saldo PASSED [ 71%]
tests/integration/test_integracao_estoque_pedidos.py::TestConcorrenca::test_cancelamentos_simultaneos_dev
olvem_uma_unica_vez PASSED [ 76%]
tests/integration/test_integracao_estoque_pedidos.py::TestFalhasDeComunicacao::test_estoque_fora_do_ar_ret
orna_503 PASSED [ 80%]
tests/integration/test_integracao_estoque_pedidos.py::TestFalhasDeComunicacao::test_estoque_que_nao_respon
de_gera_timeout_e_503 PASSED [ 85%]
tests/integration/test_integracao_estoque_pedidos.py::TestFalhasDeComunicacao::test_queda_do_estoque_mante
m_pedido_confirmado PASSED [ 90%]
tests/integration/test_integracao_estoque_pedidos.py::TestInterfaceDeLinhaDeComando::test_fluxo_completo_p
elas_duas_clis PASSED [ 95%]
tests/integration/test_integracao_estoque_pedidos.py::TestInterfaceDeLinhaDeComando::test_cli_de_pedidos_m
ostra_recusa_do_estoque PASSED [100%]

- generated xml file: /home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/junit-
integracao.xml -
- Generated html report:
file:///home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/relatorio-testes-integracao.html
-
===== 21 passed in 4.25s =====
```

relatorio-testes-integracao.html

Report generated on 24-Sep-2026 at 11:45:11 by `pytest-html` v4.1.1

Environment

Python	3.11.15
Platform	Linux-6.18.44-fc-v37-x86_64-with-glibc2.39
Packages	- pytest: 8.3.3 - pluggy: 1.6.0
Plugins	- metadata: 3.1.1 - html: 4.1.1 - cov: 5.0.0
JAVA_HOME	/usr/lib/jvm/java-21-openjdk-amd64

Summary

21 tests took 00:00:04.

(Un)check the boxes to filter the results.

☒ 0 Failed, ☒ 21 Passed, ☒ 0 Skipped, ☒ 0 Expected failures, ☒ 0 Unexpected passes, ☒ 0 Errors, ☒ 0 Reruns

[Show all details](#) / [Hide all details](#)

Result 	Test	Duration	Links
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestComunicacaoEntreServicos::test_pedidos_reconhece_estoque_online	536 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestComunicacaoEntreServicos::test_catalogo_do_pedidos_reflete_produtos_do_estoque	10 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_pedido_confirmado_baixa_saldo_no_estoque	16 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_preco_do_pedido_vem_do_estoque	9 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_desconto_progressivo_com_precos_reais_do_estoque[499.99-1-0-499.99]	8 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_desconto_progressivo_com_precos_reais_do_estoque[250.00-2-5-475.00]	8 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_desconto_progressivo_com_precos_reais_do_estoque[1000.00-1-10-900.00]	8 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_estoque_insuficiente_recusa_pedido_sem_baixa_parcial	14 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_produto_inexistente_retorna_422_sem_alterar_estoque	9 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_pedido_invalido_nao_altera_estoque	7 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_entrada_no_estoque_libera_pedido_antes_recusado	15 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCriacaoDePedido::test_pedido_que_atinge_minimo_gera_alerta_de_reposicao	11 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCancelamento::test_cancelamento_devolve_itens_ao_estoque	15 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestCancelamento::test_cancelamento_duplicado_nao_devolve_em_dobro	14 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestConcorrencia::test_pedidos_simultaneos_nao_vendem_alem_do_saldo	95 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestConcorrencia::test_cancelamentos_simultaneos_devolvem_uma_unica_vez	31 ms	
Passed	tests/integration/test_integracao_estoque_pedidos.py::TestFalhasDeComunicacao::test_estoque_fora_do_ar_retorna_503	333 ms	

Figura 3: Relatório pytest-html dos testes de integração (entrega/evidencias/relatorio-testes-integracao.html).

6.4 Execução funcional pelo terminal (CLIs)

Capturas feitas durante a gravação do vídeo, em quatro terminais reais: no alto, os servidores do Estoque (Terminal 1, porta 5001) e de Pedidos (Terminal 2, porta 5002), cujos logs mostram cada chamada HTTP recebida; embaixo, as interfaces de linha de comando usadas pelo usuário (Terminais 3 e 4).

The image displays four terminal windows arranged in a 2x2 grid. The top row shows server logs for 'Aplicação 1: servidor do Estoque' (Terminal 1) and 'Aplicação 2: servidor de Pedidos' (Terminal 2). Both show Flask app startup messages and a warning about using a development server. Terminal 1 logs show three POST requests to /api/produtos. The bottom row shows CLI interfaces for 'CLI do Estoque (usuário)' (Terminal 3) and 'CLI de Pedidos (usuário)' (Terminal 4). Terminal 3 shows a menu with options 1-4 and a table of inventory items. Terminal 4 shows a menu with options 3-5 and a confirmation message for a pending order.

```
Terminal 1 - Aplicação 1: servidor do Estoque
(.venv) aluno@projeto:~/Projeto$ python -m estoque_service
* Serving Flask app 'estoque_service.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [24/Sep/2026 11:45:33] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - - [24/Sep/2026 11:45:38] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - - [24/Sep/2026 11:45:43] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - - [24/Sep/2026 11:45:44] "GET /api/produtos HTTP/1.1" 200 -

Terminal 2 - Aplicação 2: servidor de Pedidos
(.venv) aluno@projeto:~/Projeto$ python -m pedidos_service
* Serving Flask app 'pedidos_service.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit

Terminal 3 - CLI do Estoque (usuário)
Quantidade inicial [0]: 2
Estoque mínimo [0]: 1
OK: produto MON-001 cadastrado com saldo 2.

SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

SKU      NOME      PREÇO  SALDO  MINIMO  SITUAÇÃO
-----
MON-001  Monitor 27 polegadas  R$ 1.200,00  2      1      OK
MOU-001  Mouse sem fio      R$ 120,00   5      2      OK
TEC-001  Teclado mecânico   R$ 250,00   10     3      OK

SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção:
```

```
Terminal 4 - CLI de Pedidos (usuário)
(.venv) aluno@projeto:~/Projeto$

3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 2

Cliente: Maria Souza
Informe os itens (SKU em branco para finalizar).
SKU: TEC-001
Quantidade: 2
SKU: MOU-001
Quantidade: 1
SKU:
OK: pedido confirmado!
Pedido #1 - Maria Souza - CONFIRMADO
TEC-001 Teclado mecânico      2 x R$ 250,00 = R$ 500,00
MOU-001 Mouse sem fio        1 x R$ 120,00 = R$ 120,00
Subtotal R$ 620,00 | Desconto 5% (-R$ 31,00) | TOTAL R$ 589,00

SERVIÇO DE PEDIDOS (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção:
```

Figura 4: Servidores nos Terminais 1 e 2. Três produtos cadastrados pela CLI do Estoque (Terminal 3); o log do Terminal 1 registra cada POST /api/produtos.

The image displays four terminal windows arranged in a 2x2 grid. The top row shows server logs for 'Aplicação 1: servidor do Estoque' (Terminal 1) and 'Aplicação 2: servidor de Pedidos' (Terminal 2). Terminal 1 logs show a POST request to /api/estoque/baixas. Terminal 2 logs show a GET request to /api/catalogo and a POST request to /api/pedidos. The bottom row shows CLI interfaces for 'CLI do Estoque (usuário)' (Terminal 3) and 'CLI de Pedidos (usuário)' (Terminal 4). Terminal 3 shows a table of inventory items with updated stock levels. Terminal 4 shows a confirmation message for a completed order and a menu with options 1-5.

```
Terminal 1 - Aplicação 1: servidor do Estoque
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5001
Press CTRL+C to quit
127.0.0.1 - - [24/Sep/2026 11:45:33] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - - [24/Sep/2026 11:45:38] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - - [24/Sep/2026 11:45:43] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - - [24/Sep/2026 11:45:44] "GET /api/produtos HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 11:45:50] "GET /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 11:46:01] "GET /api/produtos HTTP/1.1" 200 -

Terminal 2 - Aplicação 2: servidor de Pedidos
(.venv) aluno@projeto:~/Projeto$ python -m pedidos_service
* Serving Flask app 'pedidos_service.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
127.0.0.1 - - [24/Sep/2026 11:45:50] "GET /api/catalogo HTTP/1.1" 200 -
127.0.0.1 - - [24/Sep/2026 11:45:58] "POST /api/pedidos HTTP/1.1" 201 -

Terminal 3 - CLI do Estoque (usuário)
MON-001  Monitor 27 polegadas  R$ 1.200,00  2      1      OK
MOU-001  Mouse sem fio      R$ 120,00   5      2      OK
TEC-001  Teclado mecânico   R$ 250,00   10     3      OK

SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

SKU      NOME      PREÇO  SALDO  MINIMO  SITUAÇÃO
-----
MON-001  Monitor 27 polegadas  R$ 1.200,00  2      1      OK
MOU-001  Mouse sem fio      R$ 120,00   4      2      OK
TEC-001  Teclado mecânico   R$ 250,00   8      3      OK

SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

Terminal 4 - CLI de Pedidos (usuário)
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 2

Cliente: Maria Souza
Informe os itens (SKU em branco para finalizar).
SKU: TEC-001
Quantidade: 2
SKU: MOU-001
Quantidade: 1
SKU:
OK: pedido confirmado!
Pedido #1 - Maria Souza - CONFIRMADO
TEC-001 Teclado mecânico      2 x R$ 250,00 = R$ 500,00
MOU-001 Mouse sem fio        1 x R$ 120,00 = R$ 120,00
Subtotal R$ 620,00 | Desconto 5% (-R$ 31,00) | TOTAL R$ 589,00

SERVIÇO DE PEDIDOS (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção:
```

Figura 5: Pedido de R\$ 620,00 criado na CLI de Pedidos (Terminal 4) com 5% de desconto. O Terminal 1 mostra a chamada POST /api/estoque/baixas feita pelo Pedidos e o Terminal 3 mostra o saldo baixado.

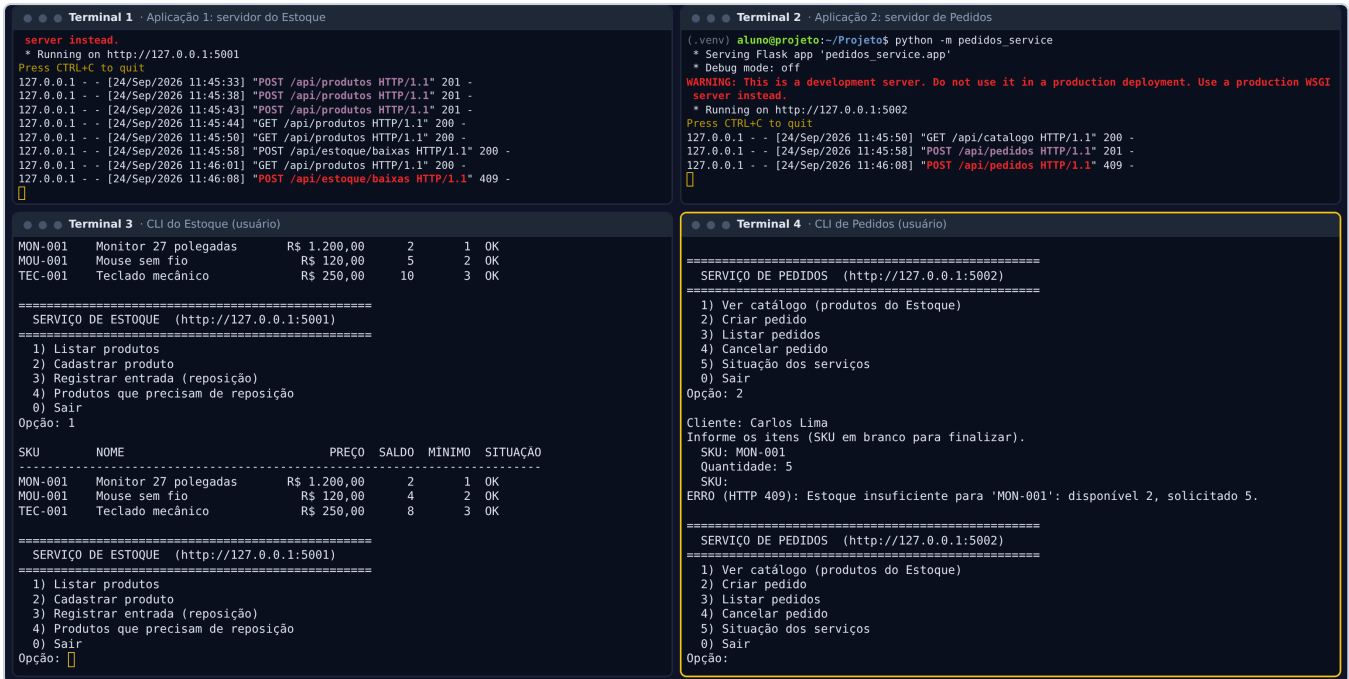


Figura 6: Pedido de 5 monitores recusado (saldo 2): HTTP 409 no Estoque (Terminal 1) propagado ao Pedidos e exibido na CLI.

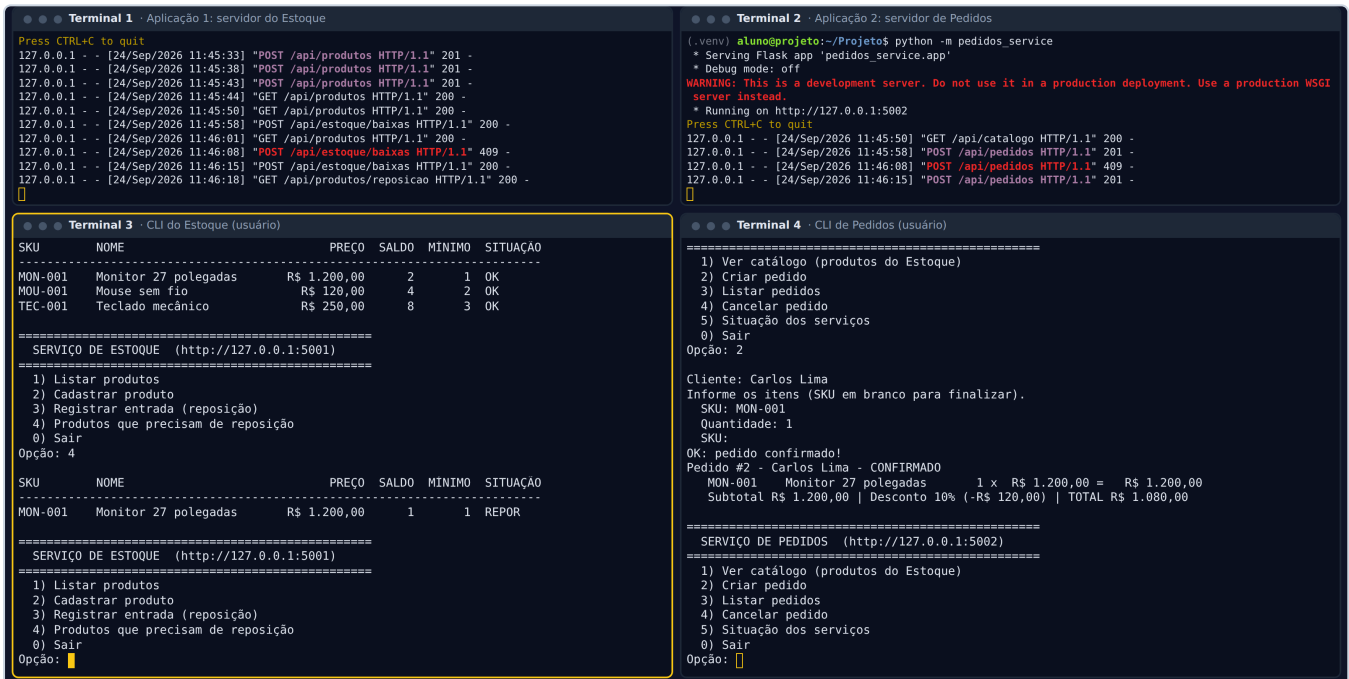


Figura 7: Pedido de R\$ 1.200,00 com 10% de desconto; na CLI do Estoque, a opção 4 mostra o monitor com situação REPOR.

```
Terminal 1 - Aplicação 1: servidor do Estoque
127.0.0.1 - [24/Sep/2026 11:45:38] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:45:43] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:45:44] "GET /api/produtos HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:45:50] "GET /api/produtos HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:45:58] "POST /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:01] "GET /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:08] "POST /api/estoque/baixas HTTP/1.1" 409 -
127.0.0.1 - [24/Sep/2026 11:46:15] "POST /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:18] "GET /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:22] "POST /api/estoque/devolucoes HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:24] "GET /api/estoque/devolucoes HTTP/1.1" 200 -
^C(.venv) aluno@projeto:~/Projetos

Terminal 3 - CLI do Estoque (usuário)
=====
SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

=====
SKU      NOME      PREÇO  SALDO  MINIMO  SITUAÇÃO
-----
MON-001  Monitor 27 polegadas  R$ 1.200,00  1      1  REPOR
MOU-001  Mouse sem fio  R$ 120,00  5      2  OK
TEC-001  Teclado mecânico  R$ 250,00  10     3  OK

=====
SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

Terminal 2 - Aplicação 2: servidor de Pedidos
* Serving Flask app 'pedidos_service.app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
127.0.0.1 - [24/Sep/2026 11:45:50] "GET /api/catalogo HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:45:58] "POST /api/pedidos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:46:08] "POST /api/pedidos HTTP/1.1" 409 -
127.0.0.1 - [24/Sep/2026 11:46:15] "POST /api/pedidos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:46:22] "POST /api/pedidos HTTP/1.1" 200 -

Terminal 4 - CLI de Pedidos (usuário)
OK: pedido confirmado!
Pedido #2 - Carlos Lima - CONFIRMADO
MON-001  Monitor 27 polegadas  1 x R$ 1.200,00 = R$ 1.200,00
Subtotal R$ 1.200,00 | Desconto 10% (-R$ 120,00) | TOTAL R$ 1.080,00

=====
SERVIÇO DE PEDIDOS (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 4

Número do pedido: 1
OK: pedido #1 cancelado; itens devolvidos ao Estoque.

=====
SERVIÇO DE PEDIDOS (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 1
```

Figura 8: Pedido #1 cancelado na CLI de Pedidos: POST /api/estoque/devolucoes no Terminal 1 e saldos restaurados (Teclado 10, Mouse 5).

```
Terminal 1 - Aplicação 1: servidor do Estoque
127.0.0.1 - [24/Sep/2026 11:45:38] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:45:43] "POST /api/produtos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:45:44] "GET /api/produtos HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:45:50] "GET /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:01] "GET /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:08] "POST /api/estoque/baixas HTTP/1.1" 409 -
127.0.0.1 - [24/Sep/2026 11:46:15] "POST /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:18] "GET /api/estoque/baixas HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:22] "POST /api/estoque/devolucoes HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:24] "GET /api/estoque/devolucoes HTTP/1.1" 200 -
^C(.venv) aluno@projeto:~/Projetos

Terminal 3 - CLI do Estoque (usuário)
=====
SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

=====
SKU      NOME      PREÇO  SALDO  MINIMO  SITUAÇÃO
-----
MON-001  Monitor 27 polegadas  R$ 1.200,00  1      1  REPOR
MOU-001  Mouse sem fio  R$ 120,00  5      2  OK
TEC-001  Teclado mecânico  R$ 250,00  10     3  OK

=====
SERVIÇO DE ESTOQUE (http://127.0.0.1:5001)
=====
1) Listar produtos
2) Cadastrar produto
3) Registrar entrada (reposição)
4) Produtos que precisam de reposição
0) Sair
Opção: 1

Terminal 2 - Aplicação 2: servidor de Pedidos
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5002
Press CTRL+C to quit
127.0.0.1 - [24/Sep/2026 11:45:50] "GET /api/catalogo HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:45:58] "POST /api/pedidos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:46:08] "POST /api/pedidos HTTP/1.1" 409 -
127.0.0.1 - [24/Sep/2026 11:46:15] "POST /api/pedidos HTTP/1.1" 201 -
127.0.0.1 - [24/Sep/2026 11:46:22] "POST /api/pedidos/1/cancelamento HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:30] "GET /health HTTP/1.1" 200 -
127.0.0.1 - [24/Sep/2026 11:46:35] "POST /api/pedidos HTTP/1.1" 503 -

Terminal 4 - CLI de Pedidos (usuário)
=====
SERVIÇO DE PEDIDOS (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 2

Cliente: Ana Costa
Informe os itens (SKU em branco para finalizar).
SKU: MOU-001
Quantidade: 1
SKU:
ERRO (HTTP 503): Não foi possível conectar ao Serviço de Estoque.

=====
SERVIÇO DE PEDIDOS (http://127.0.0.1:5002)
=====
1) Ver catálogo (produtos do Estoque)
2) Criar pedido
3) Listar pedidos
4) Cancelar pedido
5) Situação dos serviços
0) Sair
Opção: 1
```

Figura 9: Servidor do Estoque encerrado com Ctrl+C (Terminal 1): o Pedidos continua no ar e recusa o pedido com HTTP 503.

2. Cadastro do produto TEC-001 no Estoque 201 CREATED

```
>>> POST http://127.0.0.1:5001/api/produtos
{"sku": "TEC-001", "nome": "Teclado mecânico", "preco": "250.00", "quantidade": 10, "estoque_minimo": 3}
<<< {"sku": "TEC-001", "nome": "Teclado mecânico", "preco": "250.00", "quantidade": 10, "estoque_minimo": 3, "precisa_reposicao": false}
```

3. Cadastro do produto MOU-001 no Estoque 201 CREATED

```
>>> POST http://127.0.0.1:5001/api/produtos
{"sku": "MOU-001", "nome": "Mouse sem fio", "preco": "120.00", "quantidade": 5, "estoque_minimo": 2}
<<< {"sku": "MOU-001", "nome": "Mouse sem fio", "preco": "120.00", "quantidade": 5, "estoque_minimo": 2, "precisa_reposicao": false}
```

4. Cadastro do produto MON-001 no Estoque 201 CREATED

```
>>> POST http://127.0.0.1:5001/api/produtos
{"sku": "MON-001", "nome": "Monitor 27 polegadas", "preco": "1200.00", "quantidade": 2, "estoque_minimo": 1}
<<< {"sku": "MON-001", "nome": "Monitor 27 polegadas", "preco": "1200.00", "quantidade": 2, "estoque_minimo": 1, "precisa_reposicao": false}
```

5. Pedidos verifica a dependência (Estoque online) 200 OK

```
>>> GET http://127.0.0.1:5002/health
<<< {"servico": "pedidos", "status": "ok", "dependencias": {"estoque": "ok"}}
```

6. Catálogo exibido pelo Pedidos (vem do Estoque) 200 OK

```
>>> GET http://127.0.0.1:5002/api/catalogo
<<< [{"sku": "MON-001", "nome": "Monitor 27 polegadas", "preco": "1200.00", "quantidade": 2, "estoque_minimo": 1, "precisa_reposicao": false}, {"sku": "MOU-001", "nome": "Mouse sem fio", "preco": "120.00", "quantidade": 5, "estoque_minimo": 2, "precisa_reposicao": false}, {"sku": "TEC-001", "nome": "Teclado mecânico", "preco": "250.00", "quantidade": 10, "estoque_minimo": 3, "precisa_reposicao": false}]
```

7. Pedido com subtotal R\$ 620,00 → desconto de 5% 201 CREATED

```
>>> POST http://127.0.0.1:5002/api/pedidos
{"cliente": "Maria Souza", "itens": [{"sku": "TEC-001", "quantidade": 2}, {"sku": "MOU-001", "quantidade": 1}]}
<<< {"id": 1, "cliente": "Maria Souza", "status": "CONFIRMADO", "itens": [{"sku": "TEC-001", "nome": "Teclado mecânico", "quantidade": 2, "preco_unitario": "250.00", "total": "500.00"}, {"sku": "MOU-001", "nome": "Mouse sem fio", "quantidade": 1, "preco_unitario": "120.00", "total": "120.00"}], "subtotal": "620.00", "percentual_desconto": 5, "desconto": "31.00", "total": "589.00", "criado_em": "2026-09-24T11:45:13.043942+00:00", "cancelado_em": null}
```

8. Saldos no Estoque após o pedido (baixa automática) 200 OK

```
>>> GET http://127.0.0.1:5001/api/produtos
<<< [{"sku": "MON-001", "nome": "Monitor 27 polegadas", "preco": "1200.00", "quantidade": 2, "estoque_minimo": 1, "precisa_reposicao": false}, {"sku": "MOU-001", "nome": "Mouse sem fio", "preco": "120.00", "quantidade": 4, "estoque_minimo": 2, "precisa_reposicao": false}, {"sku": "TEC-001", "nome": "Teclado mecânico", "preco": "250.00", "quantidade": 8, "estoque_minimo": 3, "precisa_reposicao": false}]
```

9. Pedido acima do saldo → recusado (409) 409 CONFLICT

```
>>> POST http://127.0.0.1:5002/api/pedidos
{"cliente": "Carlos Lima", "itens": [{"sku": "MON-001", "quantidade": 5}]}
<<< {"erro": "Estoque insuficiente para 'MON-001': disponível 2, solicitado 5.", "codigo": "ESTOQUE_INSUFICIENTE"}
```

10. Pedido com produto inexistente → 422 422 UNPROCESSABLE ENTITY

```
>>> POST http://127.0.0.1:5002/api/pedidos
{"cliente": "Carlos Lima", "itens": [{"sku": "XYZ-999", "quantidade": 1}]}
<<< {"erro": "Produto 'XYZ-999' não encontrado.", "codigo": "PRODUTO_INEXISTENTE"}
```

11. Pedido com quantidade acima do limite (101) → 400 400 BAD REQUEST

```
>>> POST http://127.0.0.1:5002/api/pedidos
{"cliente": "Carlos Lima", "itens": [{"sku": "TEC-001", "quantidade": 101}]}
<<< {"erro": "A quantidade de cada item deve estar entre 1 e 100.", "codigo": "DADOS_INVALIDOS"}
```

12. Pedido de R\$ 1.200,00 → desconto de 10% 201 CREATED

```
>>> POST http://127.0.0.1:5002/api/pedidos
{"cliente": "Carlos Lima", "itens": [{"sku": "MON-001", "quantidade": 1}]}
<<< {"id": 2, "cliente": "Carlos Lima", "status": "CONFIRMADO", "itens": [{"sku": "MON-001", "nome": "Monitor 27 polegadas", "quantidade": 1, "preco_unitario": "1200.00", "total": "1200.00"}], "subtotal": "1200.00", "percentual_desconto": 10, "desconto": "120.00", "total": "1080.00", "criado_em": "2026-09-24T11:45:13.063713+00:00", "cancelado_em": null}
```

13. Produtos que precisam de reposição (saldo ≤ mínimo) 200 OK

```
>>> GET http://127.0.0.1:5001/api/produtos/reposicao
<<< [{"sku": "MON-001", "nome": "Monitor 27 polegadas", "preco": "1200.00", "quantidade": 1, "estoque_minimo": 1, "precisa_reposicao": true}]
```

14. Cancelamento do pedido 1 200 OK

```
>>> POST http://127.0.0.1:5002/api/pedidos/1/cancelamento
<<< {"id": 1, "cliente": "Maria Souza", "status": "CANCELADO", "itens": [{"sku": "TEC-001", "nome": "Teclado mecânico", "quantidade": 2, "preco_unitario": "250.00", "total": "500.00"}, {"sku": "MOU-001", "nome": "Mouse sem fio", "quantidade": 1, "preco_unitario": "120.00", "total": "120.00"}], "subtotal": "620.00", "percentual_desconto": 5, "desconto": "31.00", "total": "589.00", "criado_em": "2026-09-24T11:45:13.043942+00:00", "cancelado_em": "2026-09-24T11:45:13.070846+00:00"}
```

15. Segundo cancelamento do pedido 1 → 409 409 CONFLICT

```
>>> POST http://127.0.0.1:5002/api/pedidos/1/cancelamento
<<< {"erro": "0 pedido 1 não pode ser cancelado (status CANCELADO).", "codigo": "TRANSICAO_INVALIDA"}
```

16. Saldos no Estoque após o cancelamento (itens devolvidos) 200 OK

```
>>> GET http://127.0.0.1:5001/api/produtos
<<< [{"sku": "MON-001", "nome": "Monitor 27 polegadas", "preco": "1200.00", "quantidade": 1, "estoque_minimo": 1, "precisa_reposicao": true}, {"sku": "MOU-001", "nome": "Mouse sem fio", "preco": "120.00", "quantidade": 5, "estoque_minimo": 2, "precisa_reposicao": false}, {"sku": "TEC-001", "nome": "Teclado mecânico", "preco": "250.00", "quantidade": 10, "estoque_minimo": 3, "precisa_reposicao": false}]
```

7 Matriz de rastreabilidade

Documento gerado automaticamente por `scripts/gerar_documentacao_testes.py`. Os IDs referem-se ao catálogo em `04-casos-de-teste.md`.

30 de 30 requisitos possuem verificação automatizada.

Requisito	Descrição	Testes unitários	Testes de integração
RF-E01	Cadastrar produto com SKU, nome, preço, quantidade inicial e estoque mínimo.	UT-E18, UT-E19, UT-E20, UT-E30, UT-E40, UT-E58, UT-E76	IT02
RF-E02	Listar os produtos e consultar um produto pelo SKU.	UT-E28, UT-E30, UT-E32, UT-E33, UT-E40, UT-E42, UT-E62, UT-E63, UT-E76	n/a
RF-E03	Registrar entrada (reposição) de mercadoria em um produto.	UT-E22, UT-E44, UT-E45, UT-E46, UT-E64, UT-E65, UT-E79, UT-E80	IT09
RF-E04	Baixar (reservar) itens em lote, informando nome e preço vigente de cada item.	UT-E25, UT-E26, UT-E47, UT-E48, UT-E67, UT-E69, UT-P28	IT03, IT18
RF-E05	Devolver itens ao estoque (estorno de pedido cancelado).	UT-E53, UT-E54, UT-E55, UT-E70, UT-P29, UT-P50	IT11
RF-E06	Listar os produtos que precisam de reposição.	UT-E56, UT-E66, UT-E81	IT10
RF-E07	Oferecer uma interface de linha de comando (menu no terminal) para cadastrar, listar, registrar entradas e consultar a reposição, exibindo os erros da API ao usuário.	UT-E76, UT-E77, UT-E78, UT-E79, UT-E80, UT-E81, UT-E82, UT-E83, UT-E84, UT-E85	IT18
RN-E01	O SKU é obrigatório, único, tem de 3 a 20 caracteres (A–Z, 0–9 e hífen, sem começar com hífen) e é normalizado para maiúsculas.	UT-E01, UT-E02, UT-E03, UT-E04, UT-E18, UT-E31, UT-E41, UT-E42, UT-E43, UT-E60	n/a
RN-E02	O nome é obrigatório e tem no máximo 100 caracteres.	UT-E05, UT-E06, UT-E07, UT-E08, UT-E18	n/a
RN-E03	O preço deve ser numérico, maior que zero e no máximo R\$ 1.000.000,00, arredondado para 2 casas (half-up).	UT-E09, UT-E10, UT-E11, UT-E12, UT-E13, UT-E18, UT-E59, UT-E77	n/a
RN-E04	Quantidades por operação são inteiras entre 1 e 10.000. O saldo nunca fica negativo e a capacidade máxima por produto é 100.000 unidades.	UT-E14, UT-E15, UT-E16, UT-E17, UT-E23, UT-E24, UT-E26, UT-E27, UT-E39, UT-E45, UT-E55, UT-E65, UT-E68	n/a
RN-E05	Um produto precisa de reposição quando o saldo é menor ou igual ao estoque mínimo.	UT-E21, UT-E48, UT-E56, UT-E66, UT-E81	IT10
RN-E06	Baixas e devoluções em lote são atômicas (tudo ou nada), aceitam até 50 itens e somam SKUs repetidos.	UT-E34, UT-E35, UT-E36, UT-E37, UT-E38, UT-E39, UT-E49, UT-E50, UT-E51, UT-E54	IT06
RF-P01	Exibir o catálogo de produtos, obtido do Serviço de Estoque.	UT-P27, UT-P37, UT-P49, UT-P55, UT-P56, UT-P71, UT-P72	IT02
RF-P02	Criar pedido informando cliente e itens (SKU e quantidade).	UT-P15, UT-P16, UT-P17, UT-P21, UT-P42, UT-P57, UT-P74	IT03, IT09, IT18

Requisito	Descrição	Testes unitários	Testes de integração
RF-P03	Listar os pedidos e consultar um pedido pelo número.	UT-P21, UT-P25, UT-P40, UT-P41, UT-P47, UT-P48, UT-P62, UT-P63, UT-P77	n/a
RF-P04	Cancelar pedido, devolvendo os itens ao Serviço de Estoque.	UT-P22, UT-P23, UT-P29, UT-P50, UT-P53, UT-P64, UT-P78, UT-P79	IT11
RF-P05	Informar a saúde do serviço e a disponibilidade do Serviço de Estoque.	UT-P30, UT-P39, UT-P49, UT-P54, UT-P80	IT01, IT15
RF-P06	Oferecer uma interface de linha de comando (menu no terminal) para ver o catálogo, criar, listar e cancelar pedidos e consultar a situação dos serviços.	UT-P71, UT-P72, UT-P73, UT-P74, UT-P75, UT-P76, UT-P77, UT-P78, UT-P79, UT-P80, UT-P81, UT-P82, UT-P83, UT-P84	IT18, IT19
RN-P01	O cliente é obrigatório (3 a 80 caracteres). O pedido tem de 1 a 20 produtos distintos, com quantidade inteira de 1 a 100 por produto. Itens com o mesmo SKU são agrupados.	UT-P02, UT-P03, UT-P04, UT-P05, UT-P06, UT-P07, UT-P08, UT-P09, UT-P10, UT-P11, UT-P12, UT-P13, UT-P14, UT-P15, UT-P43, UT-P44, UT-P58, UT-P75	IT08
RN-P02	O preço unitário é sempre o informado pelo Serviço de Estoque (fonte da verdade), nunca o enviado pelo cliente.	UT-E47, UT-E67, UT-P28, UT-P38, UT-P42	IT04, IT05
RN-P03	Desconto progressivo sobre o subtotal: 0% abaixo de R\$ 500,00, 5% de R\$ 500,00 a R\$ 999,99 e 10% a partir de R\$ 1.000,00, arredondado para 2 casas.	UT-P01, UT-P18, UT-P19, UT-P20, UT-P57, UT-P74, UT-P77	IT05
RN-P04	O pedido só é confirmado se o estoque reservar todos os itens. Se houver falta de saldo ou produto inexistente, nenhum pedido é registrado.	UT-E49, UT-P31, UT-P43, UT-P45, UT-P59, UT-P76	IT06, IT07, IT09, IT13, IT19
RN-P05	Somente pedidos CONFIRMADOS podem ser cancelados. Os itens são devolvidos ao estoque antes da mudança de status, e se o estoque estiver indisponível o pedido permanece CONFIRMADO.	UT-P22, UT-P24, UT-P50, UT-P51, UT-P52, UT-P64, UT-P78	IT12, IT14, IT17
RN-P06	Falhas de comunicação com o estoque (conexão recusada, timeout, erro 5xx) resultam em HTTP 503, e respostas fora do contrato resultam em HTTP 502.	UT-P32, UT-P33, UT-P34, UT-P35, UT-P36, UT-P37, UT-P38, UT-P46, UT-P52, UT-P56, UT-P60, UT-P61, UT-P73	IT15, IT16, IT17
RNF01	As APIs seguem o estilo REST com JSON. Erros usam o formato padronizado {"erro", "codigo"} e códigos HTTP adequados.	UT-E20, UT-E28, UT-E29, UT-E38, UT-E57, UT-E58, UT-E59, UT-E61, UT-E63, UT-E68, UT-E71, UT-E72, UT-E73, UT-P14, UT-P16, UT-P17, UT-P21, UT-P25, UT-P31, UT-P57, UT-P58, UT-P59, UT-P63, UT-P65, UT-P66	IT07
RNF02	Os testes unitários devem cobrir pelo menos 90% do código (linhas e ramos), verificado automaticamente.	Verificado pela execução de <code>pytest --cov -cov-fail-under=90</code> (seção de evidências).	n/a
RNF03	Chamadas do Pedidos ao Estoque têm tempo limite configurável (padrão de 3 segundos).	UT-P26, UT-P27, UT-P34, UT-P67, UT-P68	IT16
RNF04	Operações simultâneas não podem deixar o estoque negativo nem devolver itens em duplicidade (thread-safety).	UT-E52	IT13, IT14

Requisito	Descrição	Testes unitários	Testes de integração
RNF05	Portas, endereços e a URL do estoque são configuráveis por variáveis de ambiente.	UT-E74, UT-E75, UT-E85, UT-P26, UT-P67, UT-P68, UT-P69, UT-P70, UT-P84	IT01

8 Conclusão

As duas aplicações atendem aos requisitos especificados, e isso é comprovado por evidências automatizadas e reproduzíveis. Os **256 testes unitários** exercitam cada camada de forma isolada. Com o uso de dublês (stub, mock, fake e spy) e de injeção de dependência, atingiram **100% de cobertura de linhas e 100% de ramos**, acima da meta de 90%. Os **21 testes de integração** comprovam que as aplicações funcionam em conjunto em um cenário realista: processos separados, portas TCP reais e falhas de rede provocadas de propósito.

Os testes de integração verificaram comportamentos que um teste unitário sozinho não garante: a baixa em lote é atômica entre os serviços, o preço vem sempre do Estoque, pedidos simultâneos não vendem além do saldo, cancelamentos simultâneos não devolvem itens em dobro e a queda do Estoque não deixa os dados inconsistentes.

A documentação dos testes é gerada a partir do próprio código, e a matriz de rastreabilidade liga cada requisito aos testes que o verificam. Assim é possível responder, a qualquer momento, *"o que foi testado e por quê?"*. Todos os 30 requisitos possuem verificação automatizada.

Apêndice A: Saída completa dos testes unitários

```
$ pytest tests/unit -v --cov --cov-report=term-missing --cov-fail-under=90
===== test session starts =====
platform linux -- Python 3.11.15, pytest-8.3.3, pluggy-1.6.0 --
/home/user/Projeto_Teste_Requisitos_de_Software/.venv/bin/python
metadata: {'Python': '3.11.15', 'Platform': 'Linux-6.18.44-fc-v37-x86_64-with-glibc2.39', 'Packages': {'pytest': '8.3.3', 'pluggy': '1.6.0'}, 'Plugins': {'metadata': '3.1.1', 'html': '4.1.1', 'cov': '5.0.0'}, 'JAVA_HOME': '/usr/lib/jvm/java-21-openjdk-amd64'}
rootdir: /home/user/Projeto_Teste_Requisitos_de_Software
configfile: pyproject.toml
plugins: metadata-3.1.1, html-4.1.1, cov-5.0.0
collecting ... collected 256 items

tests/unit/estoque/test_api.py::TestSaude::test_health_informa_servico_ativo PASSED [ 0%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_post_produto_valido_retorna_201_com_produto PASSED [ 0%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_post_produto_invalido_retorna_400_padronizado PASSED [ 1%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_post_produto_duplicado_retorna_409 PASSED [ 1%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_corpo_que_nao_e_objeto_json_retorna_400[nao e json-text/plain] PASSED [ 1%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_corpo_que_nao_e_objeto_json_retorna_400[[1, 2]-application/json] PASSED [ 2%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_get_lista_e_consulta_por_sku PASSED [ 2%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_get_produto_inexistente_retorna_404 PASSED [ 3%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_post_entrada_atualiza_saldo PASSED [ 3%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_post_entrada_com_quantidade_zero_retorna_400 PASSED [ 3%]
tests/unit/estoque/test_api.py::TestProdutosApi::test_get_reposicao_lista_produtos_no_minimo PASSED [ 4%]
tests/unit/estoque/test_api.py::TestBaixaEDevolucaoApi::test_post_baixa_retorna_itens_com_preco PASSED [ 4%]
tests/unit/estoque/test_api.py::TestBaixaEDevolucaoApi::test_post_baixa_sem_saldo_retorna_409_com_detalhes PASSED [ 5%]
tests/unit/estoque/test_api.py::TestBaixaEDevolucaoApi::test_post_baixa_de_produto_inexistente_retorna_404 PASSED [ 5%]
tests/unit/estoque/test_api.py::TestBaixaEDevolucaoApi::test_post_devolucao_retorna_produtos_atualizados PASSED [ 5%]
tests/unit/estoque/test_api.py::TestErrosGenericos::test_rota_inexistente_retorna_404_em_json PASSED [ 6%]
tests/unit/estoque/test_api.py::TestErrosGenericos::test_metodo_nao_permitido_retorna_405_em_json PASSED [ 6%]
tests/unit/estoque/test_api.py::TestErrosGenericos::test_erro_de_negocio_nao_mapeado_retorna_400 PASSED [ 7%]
tests/unit/estoque/test_api.py::TestInicializacao::test_main_usa_host_e_porta_das_variaveis_de_ambiente PASSED [ 7%]
tests/unit/estoque/test_api.py::TestInicializacao::test_execucao_como_modulo_usa_porta_padrao_5001 PASSED [ 7%]
tests/unit/estoque/test_cli.py::TestMenu::test_cadastrar_e_listar_produto PASSED [ 8%]
tests/unit/estoque/test_cli.py::TestMenu::test_erro_da_api_e_exibido_ao_usuario PASSED [ 8%]
tests/unit/estoque/test_cli.py::TestMenu::test_quantidade_nao_numerica_nao_chama_a_api PASSED [ 8%]
tests/unit/estoque/test_cli.py::TestMenu::test_registrar_entrada PASSED [ 9%]
tests/unit/estoque/test_cli.py::TestMenu::test_registrar_entrada_invalida[TEC-001-x-ERRO: informe um n\xafamero inteiro.] PASSED [ 9%]
tests/unit/estoque/test_cli.py::TestMenu::test_registrar_entrada_invalida[NAO-EXISTE-1-ERRO (HTTP 404): Produto 'NAO-EXISTE' n\xae3o encontrado.] PASSED [ 10%]
tests/unit/estoque/test_cli.py::TestMenu::test_reposicao_lista_produtos_no_minimo PASSED [ 10%]
tests/unit/estoque/test_cli.py::TestMenu::test_opcao_invalida_e_fim_de_entrada PASSED [ 10%]
tests/unit/estoque/test_cli.py::TestMenu::test_servidor_fora_do_ar PASSED [ 11%]
tests/unit/estoque/test_cli.py::TestFormatacaoEInicializacao::test_moeda_no_formato_brasileiro[0.5-R$ 0,50] PASSED [ 11%]
tests/unit/estoque/test_cli.py::TestFormatacaoEInicializacao::test_moeda_no_formato_brasileiro[250.00-R$ 250,00] PASSED [ 12%]
tests/unit/estoque/test_cli.py::TestFormatacaoEInicializacao::test_moeda_no_formato_brasileiro[1234567.5-R$ 1.234.567,50] PASSED [ 12%]
tests/unit/estoque/test_cli.py::TestFormatacaoEInicializacao::test_execucao_como_modulo_usa_url_do_ambiente PASSED [ 12%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_normaliza_para_maiusculas_e_remove_espacos PASSED [ 13%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_aceita_skus_nos_limites_validos[ABC] PASSED [ 13%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_aceita_skus_nos_limites_validos[AAAAAAAAAAAAAAAAAAAAA] PASSED [ 14%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_aceita_skus_nos_limites_validos[1-2] PASSED [ 14%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_aceita_skus_nos_limites_validos[X9-Y8-Z7] PASSED [ 14%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[AB] PASSED [ 15%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[AAAAAAAAAAAAAAAAAAAAA] PASSED [ 15%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[] PASSED [ 16%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[TEC_001] PASSED [ 16%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[TEC 001] PASSED [ 16%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[-TEC] PASSED [ 17%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_skus_fora_do_padrao[T\x9c9C-01] PASSED [ 17%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_sku_que_nao_e_texto[None] PASSED [ 17%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_sku_que_nao_e_texto[123] PASSED [ 18%]
tests/unit/estoque/test dominio.py::TestValidarSku::test_rejeita_sku_que_nao_e_texto[valor2] PASSED [ 18%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_remove_espacos_das_pontas PASSED [ 19%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_aceita_nome_com_tamanho_maximo PASSED [ 19%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_rejeita_nome_acima_do_tamanho_maximo PASSED [ 19%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_rejeita_nome_vazio_ou_invalido[None] PASSED [ 20%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_rejeita_nome_vazio_ou_invalido[] PASSED [ 20%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_rejeita_nome_vazio_ou_invalido[ ] PASSED [ 21%]
tests/unit/estoque/test dominio.py::TestValidarNome::test_rejeita_nome_vazio_ou_invalido[42] PASSED [ 21%]
tests/unit/estoque/test dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[10-esperado0] PASSED [ 21%]
tests/unit/estoque/test dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[10-esperado1] PASSED
```

```

[ 22%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[10.5-esperado2]
PASSED [ 22%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[entrada3-esperado3]
PASSED [ 23%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[ 7.25 -esperado4]
PASSED [ 23%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[0.01-esperado5]
PASSED [ 23%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_aceita_precos_validos_e_normaliza_duas_casas[1000000.00-esperado6]
PASSED [ 24%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_arredonda_meio_centavo_para_cima PASSED [ 24%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_zero_ou_negativo[0] PASSED [ 25%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_zero_ou_negativo[0.00] PASSED [ 25%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_zero_ou_negativo[-1] PASSED [ 25%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_zero_ou_negativo[0.004] PASSED [ 26%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_acima_do_maximo PASSED [ 26%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[None] PASSED [ 26%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[True] PASSED [ 27%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[entrada2] PASSED [ 27%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[abc] PASSED [ 28%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[NaN] PASSED [ 28%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[Infinity] PASSED [ 28%]
tests/unit/estoque/test_dominio.py::TestValidarPreco::test_rejeita_preco_nao_numerico[] PASSED [ 29%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_aceita_quantidades_dentro_do_intervalo[1] PASSED [ 29%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_aceita_quantidades_dentro_do_intervalo[500] PASSED [ 30%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_aceita_quantidades_dentro_do_intervalo[10000] PASSED [ 30%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidades_fora_do_intervalo[0-no m\xedximo 1]
PASSED [ 30%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidades_fora_do_intervalo[-5-no m\xedximo 1]
PASSED [ 31%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidades_fora_do_intervalo[10001-no m\xedximo 10000]
PASSED [ 31%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidade_nao_inteira[True] PASSED [ 32%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidade_nao_inteira[1.0] PASSED [ 32%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidade_nao_inteira[5] PASSED [ 32%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_rejeita_quantidade_nao_inteira[None] PASSED [ 33%]
tests/unit/estoque/test_dominio.py::TestValidarQuantidade::test_minimo_configuravel_aceita_zero_e Informa_oCampo PASSED [ 33%]
tests/unit/estoque/test_dominio.py::TestProduto::test_criar_com_dados_validos_normaliza_campos PASSED [ 33%]
tests/unit/estoque/test_dominio.py::TestProduto::test_criar_usa_zero_como_padrao_para_saldo_e_minimo PASSED [ 34%]
tests/unit/estoque/test_dominio.py::TestProduto::test_criar_rejeita_dados_que_nao_sao_objeto[None] PASSED [ 34%]
tests/unit/estoque/test_dominio.py::TestProduto::test_criar_rejeita_dados_que_nao_sao_objeto[dados1] PASSED [ 35%]
tests/unit/estoque/test_dominio.py::TestProduto::test_criar_rejeita_dados_que_nao_sao_objeto[tex] PASSED [ 35%]
tests/unit/estoque/test_dominio.py::TestProduto::test_precisa_reposicao_quando_saldo_menor_ou_igual_ao_minimo[2-True] PASSED [ 35%]
tests/unit/estoque/test_dominio.py::TestProduto::test_precisa_reposicao_quando_saldo_menor_ou_igual_ao_minimo[3-True] PASSED [ 36%]
tests/unit/estoque/test_dominio.py::TestProduto::test_precisa_reposicao_quando_saldo_menor_ou_igual_ao_minimo[4-False]
PASSED [ 36%]
tests/unit/estoque/test_dominio.py::TestProduto::test_adicionar_incrementa_saldo PASSED [ 37%]
tests/unit/estoque/test_dominio.py::TestProduto::test_adicionar_ate_a_capacidade_maxima_e_permitido PASSED [ 37%]
tests/unit/estoque/test_dominio.py::TestProduto::test_adicionar_acima_da_capacidade_maxima_e_rejeitado_sem_alterar_saldo
PASSED [ 37%]
tests/unit/estoque/test_dominio.py::TestProduto::test_remover_decrementa_saldo PASSED [ 38%]
tests/unit/estoque/test_dominio.py::TestProduto::test_remover_exatamente_o_saldo_zera_o_estoque PASSED [ 38%]
tests/unit/estoque/test_dominio.py::TestProduto::test_remover_mais_que_o_saldo_lanca_estoque_insuficiente PASSED [ 39%]
tests/unit/estoque/test_dominio.py::TestProduto::test_para_dict_serializa_preco_como_texto_com_duas_casas PASSED [ 39%]
tests/unit/estoque/test_dominio.py::TestExecoes::test_execoes_informam_codigo_e_mensagem PASSED [ 39%]
tests/unit/estoque/test_repositorio_e_servico.py::TestRepositorio::test_adicionar_e_obter_produto PASSED [ 40%]
tests/unit/estoque/test_repositorio_e_servico.py::TestRepositorio::test_adicionar_sku_duplicado_lanca_erro PASSED [ 40%]
tests/unit/estoque/test_repositorio_e_servico.py::TestRepositorio::test_obter_sku_inexistente_lanca_erro PASSED [ 41%]
tests/unit/estoque/test_repositorio_e_servico.py::TestRepositorio::test_listar_retorna_produtos_ordenados_por_sku PASSED [ 41%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_agrupa_skus_repetidos_somando_quantidades PASSED [ 41%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_lista_vazia_ou_ausente[None] PASSED [ 42%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_lista_vazia_ou_ausente[itens1] PASSED [ 42%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_lista_vazia_ou_ausente[itens2] PASSED [ 42%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_lista_vazia_ou_ausente[TEC-001] PASSED [ 43%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_aceita_lote_com_numero_maximo_de_itens PASSED [ 43%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_lote_acima_do_numero_maximo_de_itens
PASSED [ 44%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_item_que_nao_e_objeto PASSED [ 44%]
tests/unit/estoque/test_repositorio_e_servico.py::TestNormalizarItens::test_rejeita_soma_agrupada_acima_do_limite_por_operacao
PASSED [ 44%]
tests/unit/estoque/test_repositorio_e_servico.py::TestCadastroEConsulta::test_cadastrar_produto_o_torna_disponivel_na_listagem

```

```

em PASSED [ 45%]
tests/unit/estoque/test_repositorio_e_servico.py::TestCadastroEConsulta::test_cadastrar_sku_duplicado_com_outra_grafia_e_rej
eitado PASSED [ 45%]
tests/unit/estoque/test_repositorio_e_servico.py::TestCadastroEConsulta::test_obter_normaliza_sku_informado PASSED [ 46%]
tests/unit/estoque/test_repositorio_e_servico.py::TestCadastroEConsulta::test_obter_com_sku_invalido_lanca_dados_invalidos
PASSED [ 46%]
tests/unit/estoque/test_repositorio_e_servico.py::TestEntrada::test_registrar_entrada_incrementa_saldo PASSED [ 46%]
tests/unit/estoque/test_repositorio_e_servico.py::TestEntrada::test_registrar_entrada_com_quantidade_invalida_nao_altera_sal
do[0] PASSED [ 47%]
tests/unit/estoque/test_repositorio_e_servico.py::TestEntrada::test_registrar_entrada_com_quantidade_invalida_nao_altera_sal
do[-1] PASSED [ 47%]
tests/unit/estoque/test_repositorio_e_servico.py::TestEntrada::test_registrar_entrada_com_quantidade_invalida_nao_altera_sal
do[3] PASSED [ 48%]
tests/unit/estoque/test_repositorio_e_servico.py::TestEntrada::test_registrar_entrada_com_quantidade_invalida_nao_altera_sal
do[None] PASSED [ 48%]
tests/unit/estoque/test_repositorio_e_servico.py::TestEntrada::test_registrar_entrada_em_produto_inexistente PASSED [ 48%]
tests/unit/estoque/test_repositorio_e_servico.py::TestBaixa::test_baixa_retorna_nome_preco_e_saldo_restante PASSED [ 49%]
tests/unit/estoque/test_repositorio_e_servico.py::TestBaixa::test_baixa_de_todo_o_saldo_gera_alerta_de_reposicao PASSED [
49%]
tests/unit/estoque/test_repositorio_e_servico.py::TestBaixa::test_baixa_e_atomica_quando_um_item_nao_tem_saldo PASSED [ 50%]
tests/unit/estoque/test_repositorio_e_servico.py::TestBaixa::test_baixa_e_atomica_quando_um_produto_nao_existe PASSED [ 50%]
tests/unit/estoque/test_repositorio_e_servico.py::TestBaixa::test_baixa_considera_soma_de_itens_repetidos PASSED [ 50%]
tests/unit/estoque/test_repositorio_e_servico.py::TestBaixa::test_baixas_concorrentes_nunca_deixam_saldo_negativo PASSED [
51%]
tests/unit/estoque/test_repositorio_e_servico.py::TestDevolucao::test_devolucao_incrementa_saldos PASSED [ 51%]
tests/unit/estoque/test_repositorio_e_servico.py::TestDevolucao::test_devolucao_com_produto_inexistente_nao_altera_nada
PASSED [ 51%]
tests/unit/estoque/test_repositorio_e_servico.py::TestDevolucao::test_devolucao_acima_da_capacidade_nao_altera_nada PASSED [
52%]
tests/unit/estoque/test_repositorio_e_servico.py::TestReposicao::test_lista_apenas_produtos_com_saldo_ate_o_minimo PASSED [
52%]
tests/unit/pedidos/test_api.py::TestSaude::test_health_informa_situacao_da_dependencia_estoque[True-ok] PASSED [ 53%]
tests/unit/pedidos/test_api.py::TestSaude::test_health_informa_situacao_da_dependencia_estoque[False-indisponivel] PASSED [
53%]
tests/unit/pedidos/test_api.py::TestCatalogoApi::test_catalogo_retorna_produtos_do_estoque PASSED [ 53%]
tests/unit/pedidos/test_api.py::TestCatalogoApi::test_catalogo_com_estoque_fora_do_ar_retorna_503 PASSED [ 54%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_post_pedido_valido_retorna_201_com_totais PASSED [ 54%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_post_pedido_invalido_retorna_400[corpo0] PASSED [ 55%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_post_pedido_invalido_retorna_400[corpo1] PASSED [ 55%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_post_pedido_invalido_retorna_400[None] PASSED [ 55%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_recusas_do_estoque_viram_status_http_adequados[itens0-409-
ESTOQUE_INSUFICIENTE] PASSED [ 56%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_recusas_do_estoque_viram_status_http_adequados[itens1-422-
PRODUTO_INEXISTENTE] PASSED [ 56%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_estoque_indisponivel_retorna_503 PASSED [ 57%]
tests/unit/pedidos/test_api.py::TestCriarPedidoApi::test_resposta_invalida_do_estoque_retorna_502 PASSED [ 57%]
tests/unit/pedidos/test_api.py::TestConsultaECancelamentoApi::test_listar_e_consultar_pedido PASSED [ 57%]
tests/unit/pedidos/test_api.py::TestConsultaECancelamentoApi::test_consultar_pedido_inexistente_retorna_404 PASSED [ 58%]
tests/unit/pedidos/test_api.py::TestConsultaECancelamentoApi::test_cancelar_pedido_retorna_200_e_segunda_vez_409 PASSED [
58%]
tests/unit/pedidos/test_api.py::TestErrosGenericos::test_rota_inexistente_e_metodo_nao_permitido_em_json PASSED [ 58%]
tests/unit/pedidos/test_api.py::TestErrosGenericos::test_erro_de_negocio_nao_mapeado_retorna_400 PASSED [ 59%]
tests/unit/pedidos/test_api.py::TestConfiguracaoEInicializacao::test_servico_padrao_le_url_e_timeout_do_ambiente PASSED [
59%]
tests/unit/pedidos/test_api.py::TestConfiguracaoEInicializacao::test_app_sem_servico_injetado_usa_configuracao_padrao PASSED
[ 60%]
tests/unit/pedidos/test_api.py::TestConfiguracaoEInicializacao::test_main_usa_host_e_porta_das_variaveis_de_ambiente PASSED
[ 60%]
tests/unit/pedidos/test_api.py::TestConfiguracaoEInicializacao::test_execucao_como_modulo_usa_porta_padrao_5002 PASSED [
60%]
tests/unit/pedidos/test_cli.py::TestCatalogo::test_catalogo_mostra_produtos_do_estoque PASSED [ 61%]
tests/unit/pedidos/test_cli.py::TestCatalogo::test_catalogo_vazio PASSED [ 61%]
tests/unit/pedidos/test_cli.py::TestCatalogo::test_catalogo_com_estoque_fora_do_ar PASSED [ 62%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_criar_pedido_mostra_itens_desconto_e_total PASSED [ 62%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_quantidade_invalida_ignora_item PASSED [ 62%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_pedido_sem_saldo_e_recusado PASSED [ 63%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_listar_pedidos PASSED [ 63%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_cancelar_pedido PASSED [ 64%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_cancelar_numero_invalido_ou_inexistente[abc-ERRO: informe o n\xfamero do
pedido.] PASSED [ 64%]
tests/unit/pedidos/test_cli.py::TestPedidos::test_cancelar_numero_invalido_ou_inexistente[99-ERRO (HTTP 404): Pedido 99
n\xae3o encontrado.] PASSED [ 64%]
tests/unit/pedidos/test_cli.py::TestSituacaoEFalhas::test_situacao_dos_servicos[True-ok] PASSED [ 65%]
tests/unit/pedidos/test_cli.py::TestSituacaoEFalhas::test_situacao_dos_servicos[False-indisponivel] PASSED [ 65%]
tests/unit/pedidos/test_cli.py::TestSituacaoEFalhas::test_opcao_invalida_e_fim_de_entrada PASSED [ 66%]
tests/unit/pedidos/test_cli.py::TestSituacaoEFalhas::test_servidor_de_pedidos_fora_do_ar PASSED [ 66%]
tests/unit/pedidos/test_cli.py::TestFormatacaoEInicializacao::test_moeda_no_formato_brasileiro PASSED [ 66%]
tests/unit/pedidos/test_cli.py::TestFormatacaoEInicializacao::test_execucao_como_modulo_usa_url_do_ambiente PASSED [ 67%]
tests/unit/pedidos/test_cliente_estoque.py::TestConfiguracao::test_remove_barra_final_e_cria_sessao_padrao PASSED [ 67%]
tests/unit/pedidos/test_cliente_estoque.py::TestChamadasBemSucedidas::test_listar_produtos_faz_get_com_timeout PASSED [ 67%]
tests/unit/pedidos/test_cliente_estoque.py::TestChamadasBemSucedidas::test_baixar_envia_itens_e_converte_preco_para_decimal
PASSED [ 68%]

```

Trabalho Parcial: Teste e Requisitos de Software

```

tests/unit/pedidos/test_dominio.py::TestValidarItens::test_validar_solicitacao_rejeita_corpo_que_nao_e_objeto[pedido] PASSED [ 90%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_item_calcula_total_e_serializa PASSED [ 91%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_totais_sem_desconto PASSED [ 91%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_totais_com_desconto_de_5_por_cento PASSED [ 91%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_totais_com_desconto_de_10_por_cento PASSED [ 92%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_pedido_novo_nasce_confirmado_e_serializa PASSED [ 92%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_cancelar_pedido_confirmado_registra_data PASSED [ 92%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_cancelar_sem_momento_usa_relogio_atual PASSED [ 93%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_cancelar_pedido_ja_cancelado_e_transicao_invalida PASSED [ 93%]
tests/unit/pedidos/test_dominio.py::TestPedido::test_pedido_nao_encontrado Informa id PASSED [ 94%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestRepositorioPedidos::test_ids_sao_sequenciais_e_listagem_e_do_mais_rec
nte PASSED [ 94%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestRepositorioPedidos::test_obter_id_inexistente_lanca_erro PASSED [ 94%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCriarPedido::test_cria_pedido_confirmado_com_preco_vindo_do_estoque
PASSED [ 95%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCriarPedido::test_envia_ao_estoque_itens_normalizados_e_agrupados
PASSED [ 95%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCriarPedido::test_dados_invalidos_nao_chegam_ao_estoque PASSED [ 96%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCriarPedido::test_recusa_do_estoque_nao_gera_pedido[itens0-
EstoqueInsuficiente] PASSED [ 96%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCriarPedido::test_recusa_do_estoque_nao_gera_pedido[itens1-
ProdutoInexistente] PASSED [ 96%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCriarPedido::test_estoque_indisponivel_nao_gera_pedido PASSED [ 97%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestConsultas::test_listar_e_obter_pedidos PASSED [ 97%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestConsultas::test_obter_pedido_inexistente PASSED [ 98%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestConsultas::test_catalogo_e_saude_delegam_ao_estoque PASSED [ 98%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCancelarPedido::test_cancelamento_devolve_itens_e_marca_cancelado
PASSED [ 98%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCancelarPedido::test_cancelar_duas_vezes_nao_devolve_em_dobro PASSED [ 99%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCancelarPedido::test_estoque_indisponivel_mantem_pedido_confirmado
PASSED [ 99%]
tests/unit/pedidos/test_servico_e_repositorio.py::TestCancelarPedido::test_cancelar_pedido_inexistente_nao_chama_estoque
PASSED [100%]

- generated xml file: /home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/junit-unitarios.xml -

----- coverage: platform linux, python 3.11.15-final-0 -----
Name                               Stmts   Miss Branch BrPart  Cover   Missing
-----
estoque_service/_init_.py           2       0      0      0    100%
estoque_service/_main_.py           8       0      2      0    100%
estoque_service/app.py              57       0      4      0    100%
estoque_service/cli.py              92       0     26      0    100%
estoque_service/dominio.py          99       0     28      0    100%
estoque_service/repositorio.py       18       0      2      0    100%
estoque_service/servico.py           58       0     18      0    100%
pedidos_service/_init_.py           2       0      0      0    100%
pedidos_service/_main_.py           8       0      2      0    100%
pedidos_service/app.py              49       0      0      0    100%
pedidos_service/cli.py             102       0     32      0    100%
pedidos_service/cliente_estoque.py   54       0      8      0    100%
pedidos_service/dominio.py          118       0     26      0    100%
pedidos_service/repositorio.py       22       0      0      0    100%
pedidos_service/servico.py           34       0      0      0    100%
-----
TOTAL                               723       0    148      0    100%
Coverage HTML written to dir /home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/cobertura_html
Coverage JSON written to file /home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/cobertura.json

Required test coverage of 90% reached. Total coverage: 100.00%
- Generated html report: file:///home/user/Projeto_Teste_Requisitos_de_Software/entrega/evidencias/relatorio-testes-
unitarios.html -
===== 256 passed in 2.05s =====

```

Apêndice B: Arquivos da entrega e execução

Arquivo	Conteúdo
Relatorio_Trabalho_Parcial.pdf	Este relatório (documentação + evidências).
codigo_fonte.zip	Código-fonte completo: estoque_service/, pedidos_service/, tests/, docs/, scripts/ e arquivos de configuração.
video_execucao.mp4	Vídeo da execução funcional: cadastro, pedidos com desconto, recusa por falta de saldo, alerta de reposição, cancelamento, queda do Estoque e execução dos testes com cobertura.
evidencias/saida_testes_*.txt	Saídas completas do terminal (pytest).
evidencias/relatorio-testes-*.html	Relatórios pytest-html (abrir no navegador).
evidencias/junit-*.xml	Resultados no formato JUnit (padrão de ferramentas de CI).
evidencias/cobertura_html/index.html	Relatório navegável de cobertura, linha a linha.
evidencias/capturas/	Capturas de tela usadas neste relatório.
evidencias/demonstracao_api.txt	Registro das requisições e respostas HTTP da demonstração.

Como reproduzir

```
python -m venv .venv && source .venv/bin/activate      # Windows: .venv\Scripts\activate
pip install -r requirements-dev.txt
pytest tests/unit --cov --cov-report=term-missing --cov-report=html --cov-fail-under=90
pytest tests/integration -v
python -m estoque_service    # terminal 1 → http://127.0.0.1:5001
python -m pedidos_service    # terminal 2 → http://127.0.0.1:5002

# (opcional) regerar PDF, ZIP, vídeo e evidências:
pip install -r requirements-docs.txt && playwright install chromium
python scripts/gerar_entrega.py
```

Repositório: https://github.com/hfalasco/Projeto_Teste_Requisitos_de_Software